

Java Interview Series · Part 13 · Q530–Q579

Collections Framework · Part 2

HashMap internals, LinkedHashMap, TreeMap and special maps, concurrent collections, immutable factories, collectors, and the map-based coding problems, with real measurements from JDK 25.

50	48	18	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `CL-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

50	7	48	18
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🗺️	01	HashMap Deep Dive	Q530–Q541	5
🧭	02	LinkedHashMap, TreeMap & Specialised Maps	Q542–Q547	21
🧵	03	Concurrent Collections	Q548–Q559	28
🧊	04	Immutable Collections & API Evolution	Q560–Q561	41
🌊	05	Collectors, Streams & Collections	Q562–Q569	44
🧪	06	Practical Map & Cache Problems	Q570–Q576	53
🏁	07	Performance, Mistakes & Rapid-Fire	Q577–Q579	63

© TechTribers. For personal use only. Do not redistribute.

All questions

🗺️ HashMap Deep Dive

Q530 How does HashMap work internally? 5

Q531 How is the bucket index calculated, and why is the table size always a power of two? 6

Q532 What is a hash collision, and how does HashMap deal with heavy collisions? 8

Q533 What are load factor and resizing in HashMap, and when does a resize happen? 10

Q534 How do you size a HashMap to avoid resizing? What is HashMap.newHashMap? 11

Q535 What makes a good hashCode()? 12

Q536 How does HashMap handle a null key, and why does the iteration order look random? 13

Q537 How do put, putIfAbsent, getOrDefault, computeIfAbsent, compute and merge differ? 14

Q538 What is the most efficient way to iterate a Map, and what can its views do? 16

Q539 Why can't you tell a missing key from a null value with get()? 17

Q540 What is Map.Entry, and what are the rules for setValue and the Entry implementations? 18

Q541 Is HashMap thread-safe? What goes wrong with concurrent use? 19

🧭 LinkedHashMap, TreeMap & Specialised Maps

Q542 How do you build an LRU cache with LinkedHashMap? 21

Q543 How does TreeMap work, and what does NavigableMap offer? 22

Q544 How do you detect overlapping intervals (calendar booking) with a TreeMap? 24

Q545 What is IdentityHashMap, and when does reference equality matter? 25

Q546 What is WeakHashMap, and when is it useful? 26

Q547 What is java.util.Properties, and how do you use it for configuration? 27

🧵 Concurrent Collections

Q548 How does ConcurrentHashMap work internally (Java 8 and later)? 28

Q549 Why are merge, compute and computeIfAbsent safe on ConcurrentHashMap, and a get-then-put sequence is not? 29

Q550 What are weakly consistent iterators, and why is ConcurrentHashMap.size() only an estimate? 30

Q551 What are ConcurrentHashMap's bulk operations, and how do LongAdder and newKeySet help? 32

Q552 How do CopyOnWriteArrayList and CopyOnWriteArraySet work, and when should you use them? 33

Q553 How do Collections.synchronizedList and synchronizedMap work, and what do they not protect? 34

Q554 What are the BlockingQueue implementations, and how do they differ? 35

Q555 How do you implement producer-consumer with a BlockingQueue? 36

Q556 What are ConcurrentLinkedQueue and ConcurrentLinkedDeque? 37

Q557 What are ConcurrentSkipListMap and ConcurrentSkipListSet? 38

Q558 What is DelayQueue, and where is it used? 39

Q559 How do you choose the right concurrent collection? 40

🧊 Immutable Collections & API Evolution

Q560 Why does the iteration order of Set.of and Map.of change between JVM runs? 41

Q561 What changed in the Collections API between Java 8 and Java 21? 43

© TechTribers. For personal use only. Do not redistribute.

🌊 Collectors, Streams & Collections

Q562 How does Collectors.groupingBy work, with downstream collectors and a map factory? 44

Q563 What is the difference between groupingBy and partitioningBy? 46

Q564 What are the pitfalls of Collectors.toMap? 47

Q565 How do you flatten nested collections with flatMap? 48

Q566 How do you invert a Map (swap keys and values), including duplicate values? 49

Q567 How do you get the top N elements per group? 50

Q568 How do you count occurrences of each element and find the most frequent one? 51

Q569 What pitfalls do parallel streams have on collections? 52

🧪 Practical Map & Cache Problems

Q570 Count word frequencies and return the top K words. 53

Q571 Implement your own HashMap (chaining and resizing). 54

Q572 How do you implement a multimap, a counter (bag) and a bidirectional map with plain collections? 56

Q573 Design an in-memory cache with a time-to-live (TTL). 57

Q574 Implement a sliding-window rate limiter. 59

Q575 How do you merge two maps, find their common keys and compute the differences? 61

Q576 Design a time-based key-value store (get the value as of a timestamp). 62

🏁 Performance, Mistakes & Rapid-Fire

Q577 What are the most common collection mistakes found in code reviews? 63

Q578 How much faster is HashSet.contains than ArrayList.contains, and when does it matter? 65

Q579 Rapid-fire: the classic collection differences in one table. 66

© TechTribers. For personal use only. Do not redistribute.

Section 01

HashMap Deep Dive

How HashMap really works: buckets and hashing, collisions and tree bins, resizing, good hash codes, and the API methods people misuse.

Questions 530–541 · 12 questions



Q530 CL-051

• Intermediate

How does HashMap work internally?

⚡ Quick answer

A `HashMap` is an array of **buckets** (`table`), each holding a chain of `Node(hash, key, value, next)` or, when crowded, a tree. `put` spreads the key's hash (`h ^ (h >>> 16)`), picks the bucket `(n - 1) & hash` , walks the chain replacing an equal key or appending a new node, converts long chains to trees, and resizes when `size > 0.75 × n` . `get` repeats the first two steps and walks the bucket.

What changed in Java 8: new nodes are appended at the **tail** of a chain (Java 7 inserted at the head, which could create an endless loop when two threads resized at once), long chains become red-black **tree bins**, and the hash spreading was simplified to a single shift-xor.

Other facts: `put` returns the previous value (or `null`), a `null` key is allowed (its hash is 0, so bucket 0), the table is allocated **lazily** on the first `put` , and iteration order follows bucket order, which is why it looks random and can change after a resize.

(The program above reads the private `table` and `Node.next` fields by reflection to prove the formula, which is why it needs `--add-opens java.base/java.util=ALL-UNNAMED` . Never do this in production code.)

Java

```
static Object[] table(HashMap<?, ?> m) throws Exception {                    // peek at the
private table (needs --add-opens)
    Field f = HashMap.class.getDeclaredField("table"); f.setAccessible(true); return (Object[])
f.get(m);
}
static Object next(Object node) throws Exception {
    Field f = node.getClass().getDeclaredField("next"); f.setAccessible(true); return f.get(node);
}
static int spread(Object key) { int h = key.hashCode(); return h ^ (h >>> 16); }

HashMap<String, Integer> map = new HashMap<>();
List<String> keys = List.of("hello", "world", "Aa", "BB");
for (String k : keys) map.put(k, k.length());
Object[] table = table(map);
System.out.println("table length " + table.length + ", size " + map.size());
for (String k : keys) {
    int predicted = (table.length - 1) & spread(k);                          // the formula
```

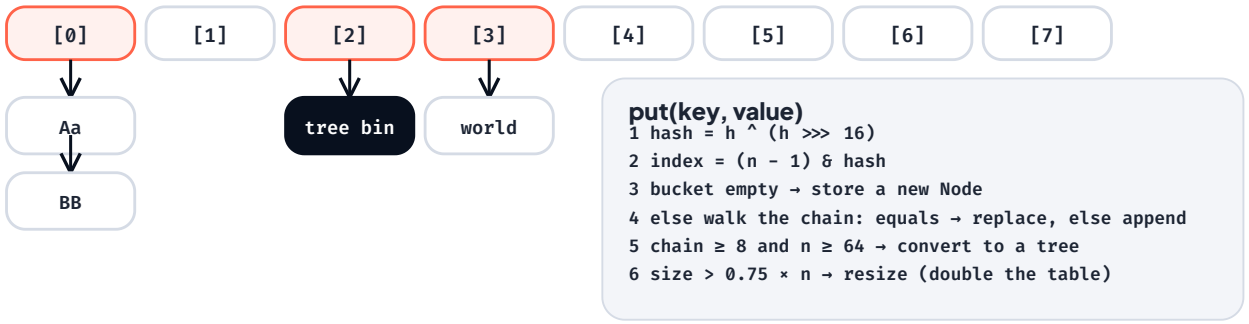

© TechTribers. For personal use only. Do not redistribute.

```
from the question
    int actual = -1;
    for (int i = 0; i < table.length; i++)
        for (Object n = table[i]; n != null; n = next(n)) if (n.toString().startsWith(k + "="))
actual = i;
    System.out.println(k + ": hash=" + k.hashCode() + " predicted bucket " + predicted + ", actual
bucket " + actual);
}
```

▶ Output

```
table length 16, size 4
hello: hash=99162322 predicted bucket 11, actual bucket 11
world: hash=113318802 predicted bucket 3, actual bucket 3
Aa: hash=2112 predicted bucket 0, actual bucket 0
BB: hash=2112 predicted bucket 0, actual bucket 0
```

HashMap = array of buckets; each bucket is a chain of Nodes (or a tree)



Node = (hash, key, value, next). get() repeats steps 1–2 and walks the bucket. A null key hashes to 0. Not thread-safe.

💡 **Pro tip**

Interview-ready summary: "array of buckets, hash → index with a mask, chaining for collisions, trees for long chains, doubling resize at load factor 0.75, and equals() decides whether two keys are the same."

Q531 CL-052 • Advanced

How is the bucket index calculated, and why is the table size always a power of two?

⚡ **Quick answer**

`index = (n - 1) & spread(hash)` where `spread(h) = h ^ (h >>> 16)`. When `n` is a power of two, `n - 1` is a mask of all ones, so the AND keeps the low bits (a fast modulo that is never negative). The shift-xor mixes the high 16 bits into the low bits, which the mask would otherwise ignore. A power-of-two size also makes resizing cheap: each node either stays at index `i` or moves to `i + oldCapacity`, decided by one extra bit.

Without the spreading step, hash codes that differ only above bit 15 (common for `Float` / `Double` values, `Integer` multiples of a power of two, or some identity hashes) would all land in the same bucket of a small table.

© TechTribers. For personal use only. Do not redistribute.

The JDK picked a cheap mix (one shift, one xor) as a compromise: the tree bins protect the worst case.

Java

```
int h = "hello".hashCode();
int spread = h ^ (h >>> 16);                                    // mix the high bits into the low
bits
System.out.println(h + " " + spread);
for (int n : new int[]{16, 32})
    System.out.println("n=" + n + " index=" + ((n - 1) & spread) + " same as modulo: " + (((n - 1) &
spread) == Math.floorMod(spread, n)));
System.out.println(Integer.toBinaryString(15) + " " + Integer.toBinaryString(31));          // the
masks

// A resize doubles n: each entry stays at index i or moves to i + oldCapacity, decided by one bit
for (String k : List.of("hello", "world", "banana", "apple")) {
    int hh = k.hashCode() ^ (k.hashCode() >>> 16);
    int oldIdx = 15 & hh, newIdx = 31 & hh;
    System.out.println(k + ": " + oldIdx + " -> " + newIdx + (newIdx == oldIdx ? " (stays)" : "
(moves +16)"));
}
// Why mixing matters: hash codes that differ only in the HIGH bits
System.out.println(((16 - 1) & 0x10000) + " " + ((16 - 1) & 0x20000) + "   after spreading: "
        + ((16 - 1) & (0x10000 ^ (0x10000 >>> 16))) + " " + ((16 - 1) & (0x20000 ^ (0x20000 >>>
16))));
```

▶ Output

```
99162322 99163451
n=16 index=11 same as modulo: true
n=32 index=27 same as modulo: true
1111 11111
hello: 11 -> 27 (moves +16)
world: 3 -> 19 (moves +16)
banana: 0 -> 0 (stays)
apple: 1 -> 17 (moves +16)
0 0   after spreading: 1 2
```

Index = (n – 1) & spread(hash). With n a power of two, the mask is just the low bits



One extra hash bit decides: 0 → same index, 1 → index + 16. Resize needs no re-hashing, just a bit test per node.

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`hash % n` can be negative for negative hash codes, and a `%` is much slower than a mask. Custom hash tables copied from textbooks often get this wrong.

Q532 CL-053 • Advanced

What is a hash collision, and how does HashMap deal with heavy collisions?

⚡ **Quick answer**

A collision is two different keys landing in the same bucket (often with the **same** `hashCode`, **like** `"Aa"` **and** `"BB"`). `HashMap` **chains them and uses** `equals` **to tell them apart. When one bucket reaches** 8 nodes**, it becomes a red-black** tree bin **if the table has at least** 64** buckets; otherwise the table is resized instead. Trees order keys with `compareTo` when keys are `Comparable`, giving O(log n) worst case. A tree reverts to a chain at 6 nodes.

Reading the output: with the default 16-bucket table, the 9th colliding key triggers a **resize** (to 32) rather than a tree, the 10th resizes again (to 64), and only the 11th finally becomes a `TreeNode` bin. With an initial capacity of 64, the 9th key converts the bucket immediately.

Measured on JDK 25, every key with `hashCode() == 42`, then a lookup of each key:

Keys	Not `Comparable` (chain scan)	`Comparable` (tree bin)
2,000	49 ms	3 ms
8,000	183 ms	16 ms
32,000	8,287 ms	9 ms

Without `Comparable`, a tree can't order the keys and has to search both subtrees, so a bad `hashCode` still degrades to O(n). This is also the basis of **hash-flooding** denial-of-service attacks: a hostile client sends thousands of keys with the same hash. Tree bins turned that from O(n²) into O(n log n) for comparable keys such as `String`.

Java

```
Map<String, Integer> m = new HashMap<>();
m.put("Aa", 1); m.put("BB", 2);                                  // different keys, SAME hashCode
System.out.println("Aa".hashCode() + " " + "BB".hashCode() + " " + "Aa".equals("BB"));
System.out.println(m.get("Aa") + " " + m.get("BB") + " " + m.size());

final class K implements Comparable<K> {                         // every instance collides on
purpose
    final int id; K(int id) { this.id = id; }
    @Override public int hashCode() { return 7; }
    @Override public boolean equals(Object o) { return o instanceof K k && k.id == id; }
    @Override public int compareTo(K o) { return Integer.compare(id, o.id); }
}
static Object[] table(HashMap<?, ?> m) throws Exception { Field f =
```


© TechTribers. For personal use only. Do not redistribute.

```
HashMap.class.getDeclaredField("table"); f.setAccessible(true); return (Object[]) f.get(m); }
static String bin(HashMap<?, ?> m) throws Exception { for (Object o : table(m)) if (o != null)
return o.getClass().getSimpleName(); return "-"; }

HashMap<K, Integer> map = new HashMap<>();
for (int i = 1; i <= 12; i++) {
    map.put(new K(i), i);
    if (i >= 8) System.out.println("keys=" + i + " table=" + table(map).length + " bin=" +
bin(map));
}
HashMap<K, Integer> roomy = new HashMap<>(64);
for (int i = 1; i <= 9; i++) roomy.put(new K(i), i);
System.out.println("initial capacity 64, keys=9 table=" + table(roomy).length + " bin=" +
bin(roomy));
```

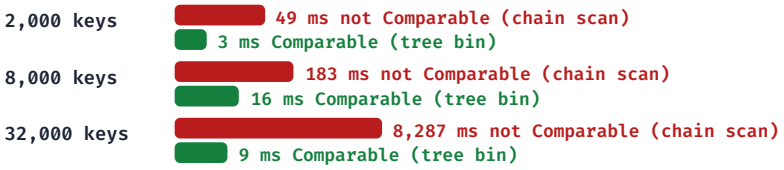
▶ Output

```
2112 2112 false
1 2 2
keys=8 table=16 bin=Node
keys=9 table=32 bin=Node
keys=10 table=64 bin=Node
keys=11 table=64 bin=TreeNode
keys=12 table=64 bin=TreeNode
initial capacity 64, keys=9 table=64 bin=TreeNode
```

A crowded bucket: chain → (resize while the table is small) → red-black tree

chain	9th node, table < 64	9th node, table ≥ 64	shrinks to ≤ 6
≤ 8 nodes	resize instead of tree	bucket → tree bin	tree → chain again

Measured: every key has hashCode() 42, then 32,000 lookups (JDK 25)



Bar length is logarithmic. A tree can order keys only if they are Comparable; otherwise it falls back to scanning.

⚠️ **Interview trap**

Tree bins are a safety net, not a licence for a poor `hashCode()`. A well-distributed hash keeps chains at 0–2 nodes; a tree bin is slower than a short chain.

© TechTribers. For personal use only. Do not redistribute.

📈 **Q533** CL-054 • Intermediate

What are load factor and resizing in HashMap, and when does a resize happen?

⚡ **Quick answer**

The **load factor** (default 0.75) sets how full the table may get: the resize **threshold** is `capacity × loadFactor`. The table is allocated lazily at the first `put` with capacity 16 (threshold 12); when `size` exceeds the threshold, a new table of **double** the capacity is allocated and every node is moved. A lower load factor wastes memory but shortens chains; a higher one saves memory but slows lookups.

Each resize is O(n): a new array is allocated and every node is relinked (no rehashing, see the index question). Growing from 16 to 256 entries involves 4 resizes and moves roughly 12 + 24 + 48 + 96 nodes. For a map you fill once and read many times, pre-sizing avoids all of that.

`HashMap(int initialCapacity, float loadFactor)` lets you tune both; the capacity is **rounded up to a power of two**. `HashSet`, `LinkedHashMap` and `ConcurrentHashMap` follow the same scheme (the latter with its own sizing control).

Java

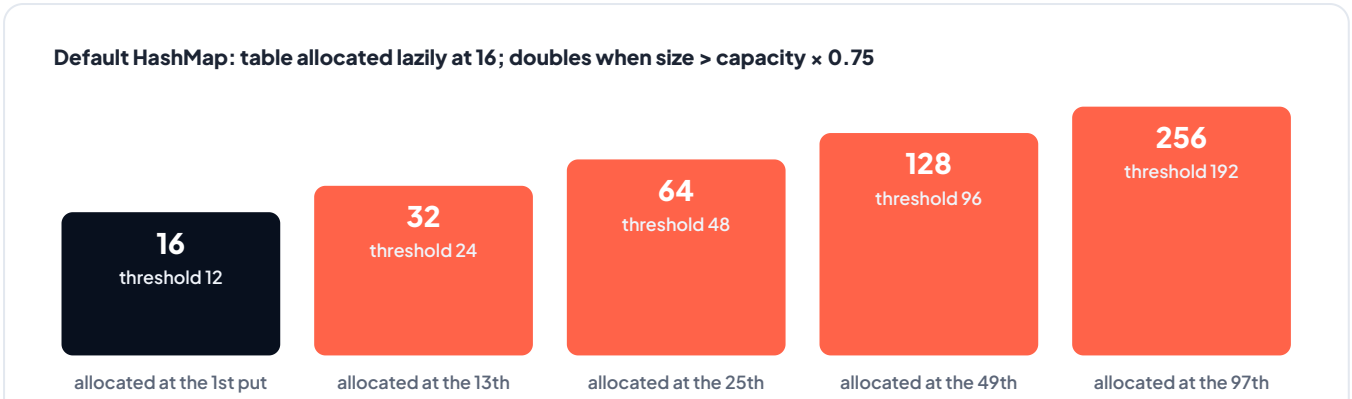
```
static Object[] table(HashMap<?, ?> m) throws Exception { Field f =
HashMap.class.getDeclaredField("table"); f.setAccessible(true); return (Object[]) f.get(m); }
static int capacity(HashMap<?, ?> m) throws Exception { Object[] t = table(m); return t == null ? 0
: t.length; }

HashMap<Integer, String> m = new HashMap<>();
System.out.println("new HashMap<>(): table length " + capacity(m) + " (allocated lazily)");
int last = capacity(m);
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 100; i++) {
    m.put(i, "v");
    int c = capacity(m);
    if (c != last) { sb.append(" size ").append(i).append("->").append(c).append(";"); last = c; }
}
System.out.println(sb);
```

▶ **Output**

```
new HashMap<>(): table length 0 (allocated lazily)
 size 1->16; size 13->32; size 25->64; size 49->128; size 97->256;
```


© TechTribers. For personal use only. Do not redistribute.



> 💡 **Pro tip**
> Interview formula: resize when `size > capacity × 0.75`. For 100 expected entries you need capacity ≥ 100 / 0.75 = 134, which rounds up to 256.

📐 **Q534** `CL-055` • Intermediate

How do you size a HashMap to avoid resizing? What is HashMap.newHashMap?

> ⚡ **Quick answer**
> `new HashMap<>(n)` means "table of about `n` buckets", not "room for `n` entries", so it resizes before reaching `n` entries (100 → 128 buckets, threshold 96). The classic fix is `(int) (n / 0.75f) + 1`. Since **Java 19**, `HashMap.newHashMap(n)` (and `HashSet.newHashSet(n)`, `LinkedHashMap.newLinkedHashMap(n)`) does that arithmetic for you and sizes the table for `n` **entries**.

`ArrayList(n)` really means room for `n` elements, but `HashMap(n)` doesn't, a long-standing inconsistency that the new factories fix. Pre-sizing matters most for large maps built in a loop (loading from a database, grouping, caching), where each avoided resize saves copying every node.

Java

```
static Object[] table(HashMap<?, ?> m) throws Exception { Field f =
HashMap.class.getDeclaredField("table"); f.setAccessible(true); return (Object[]) f.get(m); }
static int cap(HashMap<?, ?> m) throws Exception { return table(m).length; }

HashMap<Integer, Integer> a = new HashMap<>(100);                    // 100 is a table-size
request, rounded up to 128 (threshold 96)
HashMap<Integer, Integer> b = HashMap.newHashMap(100);               // Java 19+: sized for 100
ENTRIES
HashMap<Integer, Integer> c = new HashMap<>((int) (100 / 0.75f) + 1);   // the classic formula
a.put(1, 1); b.put(1, 1); c.put(1, 1);                               // the table is allocated on
the first put
System.out.println(cap(a) + " " + cap(b) + " " + cap(c));
for (int i = 2; i <= 100; i++) { a.put(i, i); b.put(i, i); c.put(i, i); }
System.out.println(cap(a) + " " + cap(b) + " " + cap(c) + "   <- a had to resize once more");
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
128 256 256
256 256 256   <- a had to resize once more
```

⚠️ **Interview trap**

Over-sizing wastes memory: the table array itself is 4–8 bytes per bucket, so `new HashMap<>(10_000_000)` allocates a ~40 MB array immediately, even if you store 5 items.

🎯 **Q535** CL-056 • Advanced

What makes a good hashCode()?

⚡ **Quick answer**

Equal objects must return equal hash codes, and unequal objects should return **different, evenly spread** ones. Combine **all fields used in equals** with a multiply-and-add scheme (`31 * h + field` , or `Objects.hash(...)` , or a record's generated `hashCode`). The classic `31` multiplier is fine for strings, but it distributes poorly when field values span a range larger than 31, as the measurement below shows.

Reading the result: `x + y` produces only 199 different values for 10,000 points, so some buckets hold 100 entries. `31 * x + y` (which is what `Objects.hash` and a record compute, plus a constant) gives 3,169 distinct values because `(x=1, y=0)` and `(x=0, y=31)` collide, yet the chains stay short (4) thanks to the spreading step and the large table. A larger multiplier separates all 10,000.

Practical rules:

- Use `Objects.hash(a, b, c)` or a record for ordinary classes: it's correct and good enough.
- Use the **same fields** as in `equals` , and never mutable fields for map keys.
- Cache an expensive hash in a `final` field if the object is immutable (`String` does exactly this).
- Never return a constant, and don't use a randomly generated value (equal objects would disagree).

Java

```
record P(int x, int y) { }
Map<String, ToIntBiFunction<Integer, Integer>> strategies = new LinkedHashMap<>();
strategies.put("x + y", (x, y) -> x + y);
strategies.put("31 * x + y", (x, y) -> 31 * x + y);
strategies.put("Objects.hash(x, y)", (x, y) -> Objects.hash(x, y));
strategies.put("record P(x, y).hashCode()", (x, y) -> new P(x, y).hashCode());
strategies.put("x * 1_000_003 + y", (x, y) -> x * 1_000_003 + y);
for (var e : strategies.entrySet()) {
    Set<Integer> distinct = new HashSet<>();
    Map<Integer, Integer> perBucket = new HashMap<>();
    for (int x = 0; x < 100; x++) for (int y = 0; y < 100; y++) {
        int h = e.getValue().applyAsInt(x, y);
        distinct.add(h);
        perBucket.merge((h ^ (h >>> 16)) & (16384 - 1), 1, Integer::sum);              // bucket in
a 16,384-slot table
    }
```

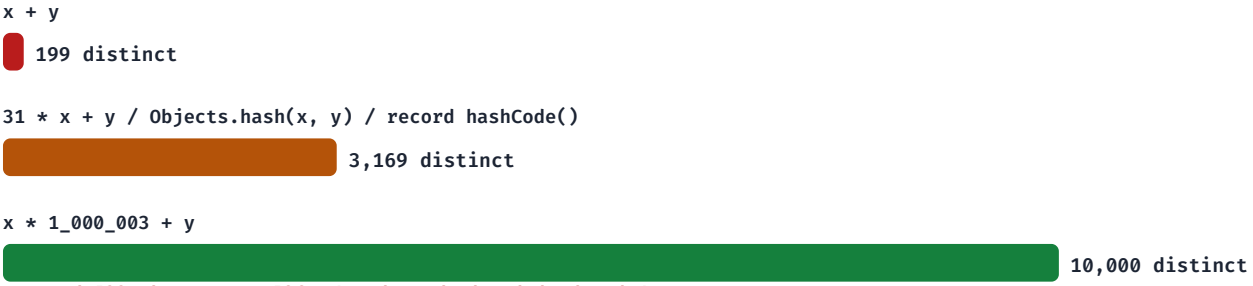

© TechTribers. For personal use only. Do not redistribute.

```
    System.out.println(String.format("%-26s distinct hash codes %5d of 10000 | longest chain %d",
e.getKey(), distinct.size(), Collections.max(perBucket.values())));
}
```

▶ Output

```
x + y                    distinct hash codes   199 of 10000 | longest chain 100
31 * x + y               distinct hash codes  3169 of 10000 | longest chain 4
Objects.hash(x, y)       distinct hash codes  3169 of 10000 | longest chain 4
record P(x, y).hashCode() distinct hash codes  3169 of 10000 | longest chain 4
x * 1_000_003 + y        distinct hash codes 10000 of 10000 | longest chain 4
```

10,000 distinct points (x, y in 0…99): how many different hash codes does each formula give?



x + y: only 199 values, so up to 100 points share a hash and a bucket chain.

31·x + y is great for Strings but weak when a field's range exceeds 31: x=1,y=0 collides with x=0,y=31.

💡 **Pro tip**

`Objects.hash(...)` boxes primitives and allocates a varargs array. In a hot path, write `31 * Integer.hashCode(x) + Integer.hashCode(y)` by hand, or use `Long.hashCode` / `Double.hashCode`.

Q536 CL-057 • Intermediate

How does HashMap handle a null key, and why does the iteration order look random?

⚡ **Quick answer**

A `null` key is allowed (one per map): its hash is defined as 0, so it lives in bucket 0. Iteration walks the table from bucket 0 upward, so the order is determined by the **hash codes and the current capacity**, not by insertion. The same keys in maps with different capacities (or after a resize) iterate in a different order. The `Map` contract promises no order at all; use `LinkedHashMap` or `TreeMap` when order matters.

Since Java 8 the iteration order of a `HashMap` for a given set of `String` keys is stable from run to run (string hash codes are specified), which tempts people to rely on it. It breaks as soon as the capacity changes, the JDK changes its hashing, or `Set.of` / `Map.of` are involved (those even randomise per JVM run, see the immutable-collections question).

© TechTribers. For personal use only. Do not redistribute.

```java
Map<String, Integer> map = new HashMap<>();
map.put(null, 0); map.put("a", 1);
System.out.println(map.get(null) + " " + map.containsKey(null) + " " + (Objects.hashCode(null) ==
0));    // the null key hashes to 0, bucket 0

List<String> keys = List.of("banana", "apple", "cherry", "date", "elderberry", "fig", "grape");
Map<String, Integer> small = new HashMap<>(), big = new HashMap<>(256);
for (String k : keys) { small.put(k, 1); big.put(k, 1); }
System.out.println(small.keySet());                                    // table of 16 buckets
System.out.println(big.keySet());                                      // same keys, table of 256
buckets: different order
System.out.println(small.equals(big));                                 // equal as maps: order is
not part of equality
Map<String, Integer> linked = new LinkedHashMap<>();
for (String k : keys) linked.put(k, 1);
System.out.println(linked.keySet());                                   // insertion order,
guaranteed
```

▶ Output

```
0 true true
[banana, date, apple, cherry, fig, grape, elderberry]
[fig, cherry, date, elderberry, banana, grape, apple]
true
[banana, apple, cherry, date, elderberry, fig, grape]
```

⚠️ **Interview trap**

Code and tests that pass only because `HashMap` happened to iterate in a convenient order are a classic source of flaky tests after a JDK upgrade. Assert on sets or sort first.

🔧 **Q537** CL-058 • Intermediate

How do put, putIfAbsent, getOrDefault, computeIfAbsent, compute and merge differ?

⚡ **Quick answer**

`put` always overwrites and returns the old value. `putIfAbsent` writes only if the key is absent (or mapped to null). `getOrDefault` reads without inserting. `computeIfAbsent(k, f)` computes and stores a value only when absent. `computeIfPresent` updates only when present. `compute` handles both cases. `merge(k, v, f)` inserts `v` if absent, otherwise stores `f(old, v)`; returning `null` from the function **removes** the key.

Method	Key absent	Key present	Returns
`put(k, v)`	insert	overwrite	old value or `null`
`putIfAbsent(k, v)`	insert	keep old	existing value or `null`

© TechTribers. For personal use only. Do not redistribute.

`computeIfAbsent(k, f)`	insert `f(k)`	keep old	current value
`computeIfPresent(k, f)`	nothing	`f(k, old)`; `null` removes	new value or `null`
`compute(k, f)`	`f(k, null)`	`f(k, old)`	new value
`merge(k, v, f)`	insert `v`	`f(old, v)`; `null` removes	new value

The classic use cases: counting with `merge(word, 1, Integer::sum)`, grouping with `computeIfAbsent(key, k -> new ArrayList<>()).add(x)`, and "update or default" with `compute`.

Java

```
Map<String, Integer> m = new HashMap<>();
System.out.println(m.put("a", 1) + " " + m.put("a", 2));                                    // put
returns the PREVIOUS value (null if none)
System.out.println(m.putIfAbsent("a", 9) + " " + m.putIfAbsent("b", 5) + " " + new TreeMap<>(m));
System.out.println(m.getOrDefault("z", -1) + " " + m.containsKey("z"));                     //
getOrDefault does NOT insert
System.out.println(m.computeIfAbsent("c", k -> 7) + " " + m.computeIfAbsent("c", k -> 99));       //
computed only while absent
System.out.println(m.computeIfPresent("a", (k, v) -> v * 10) + " " + m.computeIfPresent("zz", (k, v)
-> 1));
System.out.println(m.compute("a", (k, v) -> v == null ? 1 : v + 1) + " " + m.compute("n", (k, v) ->
v == null ? 1 : v + 1));
System.out.println(m.merge("a", 5, Integer::sum) + " " + m.merge("new", 5, Integer::sum));
System.out.println(m.merge("a", 0, (old, v) -> null) + " " + m.containsKey("a"));           // null
from the function REMOVES the key
System.out.println(m.replace("b", 50) + " " + m.replace("nope", 1) + " " + m.remove("c") + " " +
m.remove("b", 999) + " " + new TreeMap<>(m));
```

▶ Output

```
null 1
2 null {a=2, b=5}
-1 false
7 7
20 null
21 1
26 5
null false
5 null 7 false {b=50, n=1, new=5}
```

⚠️ Interview trap

Inside `computeIfAbsent` the mapping function must not modify the same map (a recursive memoisation like `fib` through `computeIfAbsent` on a `HashMap` throws `ConcurrentModificationException` since Java 9).

© TechTribers. For personal use only. Do not redistribute.

Q538 CL-059

• Basic

What is the most efficient way to iterate a Map, and what can its views do?

⚡ Quick answer

Iterate `entrySet()` when you need keys and values (one pass), `keySet()` for keys only, `values()` for values only, or `forEach((k, v) -> ...)`. Looping over `keySet()` and calling `get(k)` costs an extra lookup per key. The three views are live**: removing through a view removes from the map, but you can't `add` through `keySet()` or `values()`.

For a `HashMap`, the extra `get` per key is cheap (O(1)), so the cost is small but pointless; for a `TreeMap` each `get` is O(log n), making keySet+get noticeably slower. In Java 21, `SequencedMap` views (`sequencedKeySet()`, `sequencedEntrySet()`) add `reversed()` for ordered maps.

Java

```
Map<String, Integer> stock = new TreeMap<>(Map.of("pen", 5, "ink", 0, "pad", 2));
for (Map.Entry<String, Integer> e : stock.entrySet()) System.out.print(e.getKey() + "=" +
e.getValue() + " ");   // best: one pass
System.out.println();
for (String k : stock.keySet()) System.out.print(k + " ");
// keys only
System.out.println();
for (String k : stock.keySet()) System.out.print(stock.get(k) + " ");
// extra lookup per key: avoid
System.out.println();
int total = 0; for (int v : stock.values()) total += v;
// values only
System.out.println(total);
stock.forEach((k, v) -> System.out.print(k + ":" + v + " "));
// internal iteration
System.out.println();
System.out.println(stock.entrySet().stream().filter(e -> e.getValue() >
0).map(Map.Entry::getKey).toList());
for (Iterator<Map.Entry<String, Integer>> it = stock.entrySet().iterator(); it.hasNext(); )
    if (it.next().getValue() == 0) it.remove();
// safe removal while iterating
System.out.println(stock);
stock.keySet().remove("pad");
// views write through
stock.values().removeIf(v -> v > 100);
System.out.println(stock);
try { stock.keySet().add("new"); } catch (UnsupportedOperationException e) { System.out.println("key
views can't add"); }
```

▶ Output

```
ink=0 pad=2 pen=5
ink pad pen
0 2 5
7
ink:0 pad:2 pen:5
[pad, pen]
{pad=2, pen=5}
```


© TechTribers. For personal use only. Do not redistribute.

```
{pen=5}
key views can't add
```

> 💡 **Pro tip**
> IDEs flag "iteration over keySet() with get" for a reason. Prefer `entrySet()` or `forEach`.

? Q539 CL-060 • Intermediate

Why can't you tell a missing key from a null value with get()?

> ⚡ **Quick answer**
> `HashMap.get(key)` returns `null` both when the key is absent and when it is mapped to `null`, so use `containsKey` to tell them apart. Behaviours that depend on this: `getOrDefault` returns the **stored null** (not the default), while `computeIfAbsent` and `putIfAbsent` treat a null value as absent and overwrite it. `ConcurrentHashMap` avoids the ambiguity by **rejecting null** keys and values.

Design advice: avoid null values in maps. Store an explicit "empty" object, use `Optional` only for return values, or keep the key out of the map. If you must distinguish, `containsKey` + `get` is two lookups and **not atomic** in a concurrent setting, another reason `ConcurrentHashMap` forbids nulls.

Java

```
Map<String, String> m = new HashMap<>();
m.put("k", null);
System.out.println(m.get("k") + " " + m.get("missing"));                    // both null
System.out.println(m.containsKey("k") + " " + m.containsKey("missing"));    // the only
way to tell them apart
System.out.println(m.getOrDefault("k", "dflt") + " " + m.getOrDefault("missing", "dflt")); //
returns the stored null, not the default!
System.out.println(m.computeIfAbsent("k", x -> "computed") + " " + m);      // a null
value counts as absent here
m.put("k2", null);
System.out.println(m.putIfAbsent("k2", "filled") + " " + m.get("k2"));      // ...and here
Map<String, String> chm = new ConcurrentHashMap<>();
try { chm.put("k", null); } catch (NullPointerException e) { System.out.println("ConcurrentHashMap
rejects null: get() == null can only mean absent"); }
```

▶ **Output**

```
null null
true false
null dflt
computed {k=computed}
null filled
ConcurrentHashMap rejects null: get() == null can only mean absent
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`if (map.get(k) == null) map.put(k, v)` is the pre-Java 8 idiom and is wrong for null-valued entries and unsafe under concurrency. Use `putIfAbsent` / `computeIfAbsent`.

🏷️ **Q540** CL-061 • Intermediate

What is Map.Entry, and what are the rules for setValue and the Entry implementations?

⚡ **Quick answer**

`Map.Entry<K, V>` is a key-value pair. Entries from `entrySet()` are **live**: `setValue` writes through to the map. `Map.entry(k, v)` (Java 9) is **immutable** and rejects `null`. `AbstractMap.SimpleEntry` is mutable and allows null; `SimpleImmutableEntry` is immutable. `TreeMap.firstEntry()`, `floorEntry` and friends return read-only **snapshots**.

Entry equality is defined by key and value (`getKey().equals(...)` and `getValue().equals(...)`) and the hash code is `keyHash ^ valueHash`, so entries from different map types compare equal. Don't keep entry objects from `entrySet()` after the iteration: some maps (`IdentityHashMap`, `EnumMap`) reuse the entry object between `next()` calls, so a stored reference can silently change.

Java

```
Map<String, Integer> m = new HashMap<>(Map.of("a", 1, "b", 2));
for (Map.Entry<String, Integer> e : m.entrySet()) e.setValue(e.getValue() * 10);          //
setValue writes through to the map
System.out.println(new TreeMap<>(m));
Map.Entry<String, Integer> immutable = Map.entry("k", 1);                                  // Java
9: immutable and null-hostile
try { immutable.setValue(2); } catch (UnsupportedOperationException e) {
System.out.println("Map.entry is immutable"); }
try { Map.entry("k", null); } catch (NullPointerException e) { System.out.println("Map.entry rejects
null"); }
Map.Entry<String, Integer> simple = new AbstractMap.SimpleEntry<>("k", 1);                 //
mutable, allows null
simple.setValue(2);
System.out.println(simple + " " + simple.equals(Map.entry("k", 2)) + " " + (simple.hashCode() ==
Map.entry("k", 2).hashCode()));
TreeMap<String, Integer> tm = new TreeMap<>(Map.of("x", 1, "y", 2));
Map.Entry<String, Integer> first = tm.firstEntry();                                        // a
SNAPSHOT: not connected to the map
try { first.setValue(99); } catch (UnsupportedOperationException e) {
System.out.println("TreeMap.firstEntry() is a read-only snapshot"); }
tm.put("x", 100);
System.out.println(first + " " + tm.firstEntry());
Map.Entry<String, Integer> live = tm.entrySet().iterator().next();                         //
entrySet entries are live
live.setValue(7);
System.out.println(tm);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
{a=10, b=20}
Map.entry is immutable
Map.entry rejects null
k=2 true true
TreeMap.firstEntry() is a read-only snapshot
x=1 x=100
{x=7, y=2}
```

💡 **Pro tip**

To keep a copy of an entry beyond the iteration, use `Map.entry(e.getKey(), e.getValue())` (if values are non-null) or `new AbstractMap.SimpleEntry<>(e)`.

⚡ **Q541** CL-062 • Intermediate

Is HashMap thread-safe? What goes wrong with concurrent use?

⚡ **Quick answer**

No. Concurrent writers can **lose updates**, corrupt internal structure, return stale data, or (in Java 7) spin forever during a concurrent resize. Even `ConcurrentHashMap` doesn't make a **read-modify-write sequence** (`get` then `put`) atomic. Use atomic compound operations (`merge` , `compute` , `computeIfAbsent`) on a `ConcurrentHashMap` , or `LongAdder` values for counters.

The first two numbers change on every run (shown: one run on JDK 25; across several runs `HashMap` ended between 539,345 and 908,855 and `ConcurrentHashMap` get+put between 678,563 and 954,915). Only `merge` is exact. That is the point of the question: **individually thread-safe calls don't compose**.

In the runs I made, four threads putting **disjoint** keys into one `HashMap` did not visibly lose entries, but that is luck, not a guarantee: the behaviour is undefined, and a concurrent resize can lose or duplicate nodes or, in older JDKs, create a cycle in a bucket chain that makes `get` loop forever.

Options when a map is shared: `ConcurrentHashMap` (default choice), `Collections.synchronizedMap` (every call locked; iteration needs manual locking), or confine the map to one thread and pass messages.

Java

```
static void race(String name, Map<String, Integer> map, boolean atomic) throws Exception {
    int threads = 4, perThread = 500_000;
    ExecutorService pool = Executors.newFixedThreadPool(threads);
    CyclicBarrier startTogether = new CyclicBarrier(threads);              // make the threads
really overlap
    List<Callable<Void>> jobs = new ArrayList<>();
    for (int t = 0; t < threads; t++) jobs.add(() -> {
        startTogether.await();
        for (int i = 0; i < perThread; i++) {
            if (atomic) map.merge("hits", 1, Integer::sum);                 // ONE atomic operation
            else { Integer c = map.get("hits"); map.put("hits", c == null ? 1 : c + 1); }   // read,
add, write: three steps
        }
```


© TechTribers. For personal use only. Do not redistribute.

```
            return null;
        });
        pool.invokeAll(jobs); pool.shutdown();
        System.out.println(name + ": expected " + threads * perThread + ", got " + map.get("hits"));
    }
    race("HashMap get+put", new HashMap<>(), false);
    race("ConcurrentHashMap get+put", new ConcurrentHashMap<>(), false);
    race("ConcurrentHashMap merge", new ConcurrentHashMap<>(), true);
```

▶ Output

```
HashMap get+put: expected 2000000, got 539345
ConcurrentHashMap get+put: expected 2000000, got 954915
ConcurrentHashMap merge: expected 2000000, got 2000000
```

Check-then-act loses updates: both threads read 5, both write 6



Measured: 4 threads × 500,000 increments of one counter (expected 2,000,000), JDK 25

`HashMap get + put`	`539,345 – 908,855`
`ConcurrentHashMap get + put`	`678,563 – 954,915`
`ConcurrentHashMap merge()`	`2,000,000 (exact)`

Thread-safe methods are not thread-safe sequences. Use an atomic compound operation: merge, compute, computeIfAbsent.

⚠ Interview trap

The famous Java 7 infinite loop was in `HashMap.resize` (head insertion reversing chains while another thread resized). Java 8 changed to tail insertion, which removed that specific loop but not the data races.

© TechTribers. For personal use only. Do not redistribute.

Section 02

LinkedHashMap, TreeMap & Specialised Maps

Ordered maps, sorted navigation, identity and weak keys, and Properties.

Questions 542–547 · 6 questions



Q542 CL-063

• Intermediate

How do you build an LRU cache with LinkedHashMap?

⚡ Quick answer

Construct `LinkedHashMap(capacity, 0.75f, true)` : the `true` selects **access order**, so every `get` / `put` of an existing key moves that entry to the end. Then override `removeEldestEntry` to return `size() > capacity` ; after each insertion the map evicts the head, the least recently used entry. It's three lines, O(1) per operation, but **not thread-safe**.

Why it works: `LinkedHashMap` is a `HashMap` plus a doubly-linked list through all entries. In access-order mode, `afterNodeAccess` relinks the touched entry to the tail; `afterNodeInsertion` calls `removeEldestEntry` and, if it returns `true` , removes the head.

Limits: no per-entry expiry, no size in bytes, no statistics, and even `get` **mutates** the map (so reads are not concurrent-safe, and iterating while another thread reads corrupts the order). For production caches use **Caffeine** (window-TinyLFU eviction, async loading, expiry, concurrency). The hand-built O(1) version with a custom linked list is covered in the coding part.

Java

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LruCache(int capacity) { super(16, 0.75f, true); this.capacity = capacity; }          // true =
access order
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() >
capacity; }
}
LruCache<String, Integer> cache = new LruCache<>(3);
cache.put("a", 1); cache.put("b", 2); cache.put("c", 3);
System.out.println(cache.keySet());
cache.get("a");                                          // an access moves "a" to the end
System.out.println(cache.keySet());
cache.put("d", 4);                                       // over capacity: the eldest ("b") is
evicted
System.out.println(cache.keySet() + " " + cache.containsKey("b"));
cache.getOrDefault("c", 0);                              // getOrDefault also counts as an access
cache.putIfAbsent("a", 99);                              // so does putIfAbsent on an existing key
```

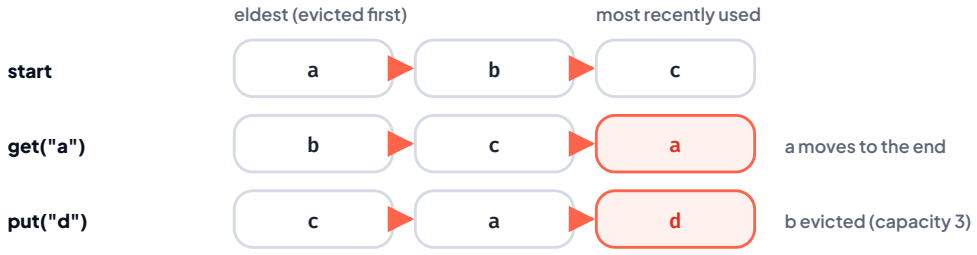

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(cache.keySet());
Map<String, Integer> threadSafe = Collections.synchronizedMap(new LruCache<>(3));      // simplest
way to share it
threadSafe.put("x", 1);
System.out.println(threadSafe);
```

▶ Output

```
[a, b, c]
[b, c, a]
[c, a, d] false
[d, c, a]
{x=1}
```

LinkedHashMap(16, 0.75f, true): access order. removeEldestEntry evicts the head



A hit relinks the node to the tail in O(1); the head is always the least recently used entry.

⚠️ **Interview trap**

With `accessOrder = true`, iterating over `keySet()` while calling `get()` on the same map throws `ConcurrentModificationException`, because each `get` is a structural modification.

🌲 **Q543** CL-064 • Intermediate

How does TreeMap work, and what does NavigableMap offer?

⚡ **Quick answer**

`TreeMap` is a **red-black tree**: always sorted by key (`compareTo` or a `Comparator`), O(log n) for `put`, `get`, `remove`. As a `NavigableMap` it adds `floorKey/Entry` (≤), `lowerKey` (<), `ceilingKey` (≥), `higherKey` (>), `firstEntry` / `lastEntry`, `pollFirstEntry` / `pollLastEntry`, `headMap` / `tailMap` / `subMap` range **views**, and `descendingMap`.

Range-view defaults: `headMap(to)` is exclusive, `tailMap(from)` inclusive, `subMap(a, b)` is `[a, b)`; the overloads with boolean flags choose inclusivity. Views are live: writes pass through, and inserting a key outside the view's range throws `IllegalArgumentException`.

`TreeMap` equality of keys is defined by the **comparator** (see the TreeSet question in Part 12): a case-insensitive comparator makes `"Key"` and `"key"` the same key. It rejects `null` keys with natural ordering, isn't thread-safe (use `ConcurrentSkipListMap`), and is slower than `HashMap` for plain lookups, but gives sorted iteration and nearest-key queries for free.

© TechTribers. For personal use only. Do not redistribute.

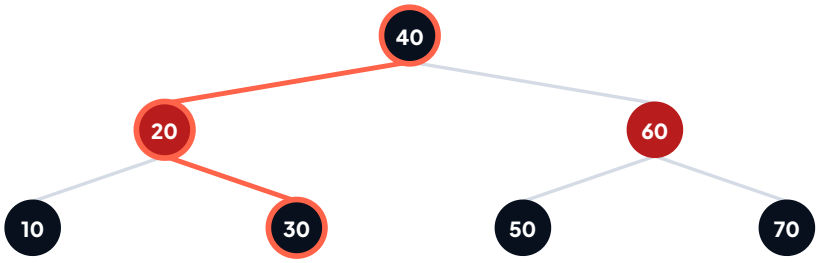
Java

```
TreeMap<Integer, String> grades = new TreeMap<>(Map.of(90, "A", 80, "B", 70, "C", 60, "D", 0, "F"));
System.out.println(grades.floorEntry(85) + " " + grades.floorKey(60) + " " + grades.ceilingKey(61) +
" " + grades.lowerKey(60) + " " + grades.higherEntry(90));
System.out.println(grades.firstKey() + " " + grades.lastEntry() + " " + grades.headMap(70) + " " +
grades.tailMap(80) + " " + grades.subMap(60, true, 80, true));
System.out.println(grades.descendingMap() + " " + grades.descendingKeySet());
Map.Entry<Integer, String> lowest = grades.pollFirstEntry();                // removes and returns
the smallest entry
System.out.println(lowest + " " + grades.size() + " " + grades.firstKey());
NavigableMap<Integer, String> top = grades.tailMap(80, false);              // a live VIEW of keys
above 80
grades.put(95, "A+");
System.out.println(top);
try { top.put(10, "x"); } catch (IllegalArgumentException e) { System.out.println("key outside the
view's range"); }
System.out.println(grades.floorEntry(-5));                                   // null when nothing
qualifies
```

▶ Output

```
80=B 60 70 0 null
0 90=A {0=F, 60=D} {80=B, 90=A} {60=D, 70=C, 80=B}
{90=A, 80=B, 70=C, 60=D, 0=F} [90, 80, 70, 60, 0]
0=F 4 60
{90=A, 95=A+}
key outside the view's range
null
```

TreeMap = red-black tree: height ≤ 2·log₂(n + 1), every operation O(log n)



floorKey(35)

40 > 35 → go left
20 < 35 → go right
30 ≤ 35, no right child
→ 30

Keys compare with compareTo / the Comparator (never equals). Red nodes never have red children; paths have equal black counts.

💡 **Pro tip**

The grade-lookup example (`floorEntry(score)`) is the canonical use: map a number onto the last threshold that doesn't exceed it, with no if/else chain.

© TechTribers. For personal use only. Do not redistribute.

Q544 CL-065 • Advanced

How do you detect overlapping intervals (calendar booking) with a TreeMap?

⚡ Quick answer

Store bookings as `start → end` in a `TreeMap`. For a new interval `[start, end)`, check only its two neighbours: the booking with the **greatest start ≤ new start** (`floorEntry`) must end before `start`, and the booking with the **least start ≥ new start** (`ceilingEntry`) must begin at or after `end`. Each check is O(log n), so `book` is O(log n) instead of scanning all bookings.

This is LeetCode "My Calendar I". It works because bookings never overlap each other, so the intervals are sorted by both start and end, and only the immediate neighbours can conflict. The same structure answers "which meeting room is free", "merge intervals as they arrive" (with `floorEntry` / `higherEntry`) and rate-limit windows.

Java

```
class Calendar {
    private final TreeMap<Integer, Integer> booked = new TreeMap<>();            // start -> end
(end exclusive)
    boolean book(int start, int end) {
        Map.Entry<Integer, Integer> before = booked.floorEntry(start);           // the booking that
starts at or before 'start'
        Map.Entry<Integer, Integer> after = booked.ceilingEntry(start);          // the next booking
starting at or after 'start'
        if (before != null && before.getValue() > start) return false;           // it is still
running when we want to start
        if (after != null && after.getKey() < end) return false;                 // the next one
starts before we finish
        booked.put(start, end);
        return true;
    }
}
Calendar c = new Calendar();
System.out.println(c.book(10, 20) + " " + c.book(15, 25) + " " + c.book(20, 30) + " " + c.book(5,
10) + " " + c.book(5, 11) + " " + c.book(30, 40) + " " + c.book(0, 100));
```

▶ Output

```
true false true true false true false
```

⚠️ Interview trap

Treat the end as **exclusive** and be explicit about it. `[10, 20)` and `[20, 30)` don't overlap; getting `<` vs `<=` wrong is the usual bug.

© TechTribers. For personal use only. Do not redistribute.

Q545 CL-066 • Advanced

What is IdentityHashMap, and when does reference equality matter?

⚡ **Quick answer**

`IdentityHashMap` compares keys with * `==` * and uses `System.identityHashCode` instead of `equals` / `hashCode` . Two equal-but-distinct objects are two different keys. Use it when identity is the real question: serialization/copy graphs (visited-object tracking), proxies, and frameworks that attach metadata to specific instances. It uses linear probing in a single array, so it is also compact.

It intentionally violates the `Map` contract (which is defined via `equals`), and the class documentation says so. Typical correct use: tracking visited nodes in a graph or object-graph traversal with cycles (`Collections.newSetFromMap(new IdentityHashMap<>())`), so an object is processed once regardless of what its `equals` says.

Java

```
String a = new String("key"), b = new String("key");
Map<String, Integer> normal = new HashMap<>();
normal.put(a, 1); normal.put(b, 2);                                    // equals() says they are the
same key
Map<String, Integer> identity = new IdentityHashMap<>();
identity.put(a, 1); identity.put(b, 2);                                // == says they are different
keys
System.out.println(normal.size() + " " + identity.size() + " " + a.equals(b) + " " + (a == b));
System.out.println(identity.get(a) + " " + identity.get("key") + " " + identity.containsKey(b));
// the literal "key" is a THIRD object
Integer x = 1000, y = 1000;
Map<Integer, String> idm = new IdentityHashMap<>(); idm.put(x, "first"); idm.put(y, "second");
System.out.println(idm.size());                                        // two separate boxes
```

▶ **Output**

```
1 2 true false
1 null true
2
```

⚠️ **Interview trap**

Never use boxed numbers or strings as `IdentityHashMap` keys unless you really mean identity: `Integer` caching (-128 to 127) makes the behaviour inconsistent.

© TechTribers. For personal use only. Do not redistribute.

Q546 CL-067 • Advanced

What is WeakHashMap, and when is it useful?

⚡ **Quick answer**

`WeakHashMap` holds its **keys through weak references**: when a key is no longer strongly reachable anywhere else, the garbage collector may clear it and the entry disappears from the map automatically. It suits caches of metadata keyed by objects whose lifetime you don't control (class-loader keyed data, per-object listeners, canonicalising maps). The **value must not strongly reference its key**, or the entry can never be collected.

Caveats that matter in practice:

- **Never use `String` literals, boxed small ints or other interned/cached objects as keys**: they stay strongly reachable, so the entry never goes away.
- Cleanup is lazy: stale entries are expunged on the next map operation.
- It isn't thread-safe; wrap with `Collections.synchronizedMap` if shared.
- Cleared entries make `size()` and iteration reflect GC timing, so behaviour is non-deterministic by design.

For cache semantics (size limits, expiry, refresh) prefer Caffeine; reserve `WeakHashMap` for "attach data to an object for as long as it lives".

Java

```
WeakHashMap<Object, String> map = new WeakHashMap<>();
Object key = new Object();
Object strong = new Object();
map.put(key, "metadata");
map.put(strong, "kept");
System.out.println(map.size());
key = null;                                                    // drop the only strong
reference to the first key
for (int i = 0; i < 40 && map.size() > 1; i++) { System.gc(); Thread.sleep(50); }     // GC timing
is not guaranteed, so retry
System.out.println(map.size() + " " + map.containsValue("kept") + " " +
map.containsValue("metadata"));
System.out.println(strong != null);                            // keeps the second key
reachable until here
```

▶ **Output**

```
2
1 true false
true
```

💡 **Pro tip**

Pair with the reference-types question in the JVM part: weak keys are collected at the next GC, soft references only under memory pressure.

© TechTribers. For personal use only. Do not redistribute.

Q547 CL-068

• Basic

What is java.util.Properties, and how do you use it for configuration?

⚡ Quick answer

`Properties` is a `Hashtable<Object, Object>` specialised for **string** key-value pairs, with `load` / `store` for `.properties` text files (`key=value` , `#` comments, `\` escapes), `getProperty(key, default)` , and an optional **defaults** chain. It's the oldest configuration format in Java; modern code often prefers YAML or Spring's `@ConfigurationProperties` , but `Properties` remains everywhere (system properties, resource bundles, JDBC connection settings).

Quirks: `Properties` inherits `put` / `get` from `Hashtable` , so you can store non-strings and break `getProperty` ; it is synchronised (slow, legacy); the ordering is undefined; `System.getProperties()` returns a live `Properties` . The default text encoding is ISO-8859-1 for `load(InputStream)` (use a `Reader` with UTF-8 instead), a classic source of garbled non-ASCII values.

Java

```
Properties defaults = new Properties();
defaults.setProperty("timeout", "30"); defaults.setProperty("mode", "prod");
Properties p = new Properties(defaults);                                    // fallback defaults
p.load(new StringReader("# comment\nhost = db.example.com\nport=5432\ntimeout = 5\n"));
System.out.println(p.getProperty("host") + " " + p.getProperty("port") + " " +
p.getProperty("timeout") + " " + p.getProperty("mode") + " " + p.getProperty("missing") + " " +
p.getProperty("missing", "n/a"));
System.out.println(p.size() + " " + p.stringPropertyNames().stream().sorted().toList());       //
size() excludes defaults, stringPropertyNames() includes them
StringWriter out = new StringWriter();
Properties small = new Properties(); small.setProperty("a", "1"); small.store(out, null);
System.out.println(out.toString().lines().filter(l -> !l.startsWith("#")).toList());
small.put("n", 42);                                                         // legal (it is a
Hashtable<Object, Object>) but getProperty() ignores non-String values
System.out.println(small.getProperty("n") + " " + small.get("n"));
```

▶ Output

```
db.example.com 5432 5 prod null n/a
3 [host, mode, port, timeout]
[a=1]
null 42
```

⚠️ Interview trap

`Properties` values are always strings. `Integer.parseInt(p.getProperty("port"))` throws `NumberFormatException` (or NPE for a missing key) unless you validate and supply defaults.

© TechTribers. For personal use only. Do not redistribute.

Section 03

Concurrent Collections

How ConcurrentHashMap, copy-on-write lists, blocking queues and skip lists work, and how to pick one.

Questions 548–559 · 12 questions



Q548 CL-069

• Advanced

How does ConcurrentHashMap work internally (Java 8 and later)?

⚡ Quick answer

Since Java 8 it uses a single table of buckets like `HashMap` with **fine-grained concurrency**: reads are **lock-free** (volatile reads), inserting into an **empty bucket** uses a **CAS**, and inserting into an occupied bucket takes a `synchronized` lock on that bucket's **first node only**. Resizing is done cooperatively (threads help transfer buckets, leaving `ForwardingNode` s). The size is kept in striped counters like `LongAdder` , so `size()` is cheap but only an estimate while others write.

Java 7 and earlier used **segments** (16 independent locks by default, `concurrencyLevel`). Java 8 removed them: the lock granularity became one bucket, which scales with the table size, and treeification of long chains was added.

Why `null` is rejected: in a concurrent map `get(k) == null` must unambiguously mean "absent", and `containsKey` followed by `get` is not atomic, so an allowed null value could never be interpreted safely.

Iterators are **weakly consistent** (next question), and the compound methods `compute` , `computeIfAbsent` , `merge` are atomic *per key* because they run under the bucket lock.

Java

```
ConcurrentHashMap<String, Integer> m = new ConcurrentHashMap<>();
try { m.put(null, 1); } catch (NullPointerException e) { System.out.println("null key rejected"); }
try { m.put("k", null); } catch (NullPointerException e) { System.out.println("null value
rejected"); }
m.put("a", 1); m.put("b", 2);
System.out.println(m.get("a") + " " + m.getOrDefault("z", 0) + " " + m.mappingCount() + " " +
m.size());      // mappingCount() returns a long: preferred for huge maps
Set<String> keys = ConcurrentHashMap.newKeySet();                    // a concurrent Set backed
by a ConcurrentHashMap
keys.add("x"); keys.add("x");
System.out.println(keys + " " + m.keySet().getClass().getSimpleName());
```

▶ Output

```
null key rejected
null value rejected
```


© TechTribers. For personal use only. Do not redistribute.

```
1 0 2 2
[x] KeySetView
```

ConcurrentHashMap (Java 8+): no segments, lock per bucket only when needed

Reads: get()
no locking at all: volatile reads of table and node fields

Empty bucket: put()
CAS the new node in; no lock, retry on failure

Occupied bucket: put()
synchronized on the FIRST node of that bucket only

Resize
ForwardingNode marks moved buckets; threads help transfer

size() = baseCount + a striped CounterCell[] (like LongAdder): cheap under contention, but only an estimate while writers run.
Bins with ≥ 8 colliding nodes become trees, as in HashMap. null keys and values are rejected: get() == null must mean absent.

💡 **Pro tip**
Keep the functions you pass to `compute*` / `merge` **short and non-blocking**, and never touch the same map from inside them: they run while the bucket is locked.

Q549 CL-070 • Advanced

Why are merge, compute and computeIfAbsent safe on ConcurrentHashMap, and a get-then-put sequence is not?

⚡ **Quick answer**
Each plain method (`get` , `put`) is thread-safe by itself, but a sequence of two calls isn't atomic: another thread can run between them. `merge` , `compute` , `computeIfAbsent` and `putIfAbsent` perform the **whole read-modify-write under the bucket lock**, so concurrent updates to the same key are serialised. `computeIfAbsent` also guarantees the mapping function runs **at most once** per key.

The same applies to `Collections.synchronizedMap` : each call is locked, but `if (!map.containsKey(k)) map.put(k, v)` is still a race unless you lock the whole sequence yourself. With `ConcurrentHashMap` , express the whole sequence as one `compute*` / `merge` / `putIfAbsent` call. For high-contention counters, store `LongAdder` values (next question).

Java

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
int threads = 4, per = 100_000;
ExecutorService pool = Executors.newFixedThreadPool(threads);
List<Callable<Void>> jobs = new ArrayList<>();
for (int t = 0; t < threads; t++) jobs.add(() -> { for (int i = 0; i < per; i++)
counts.merge("hits", 1, Integer::sum); return null; });
pool.invokeAll(jobs); pool.shutdown();
System.out.println(counts);                                    // exactly 400000:
merge() is atomic
```


© TechTribers. For personal use only. Do not redistribute.

```
ConcurrentHashMap<String, Integer> racy = new ConcurrentHashMap<>();       // check-then-act is NOT
atomic, though every call is thread-safe
CyclicBarrier gate = new CyclicBarrier(threads);
ExecutorService pool2 = Executors.newFixedThreadPool(threads);
List<Callable<Void>> racyJobs = new ArrayList<>();
for (int t = 0; t < threads; t++) racyJobs.add(() -> { gate.await(); for (int i = 0; i < per; i++) {
Integer c = racy.get("hits"); racy.put("hits", c == null ? 1 : c + 1); } return null; });
pool2.invokeAll(racyJobs); pool2.shutdown();
System.out.println("racy result <= expected: " + (racy.get("hits") <= threads * per));       //
usually far below 400000

AtomicInteger created = new AtomicInteger();
ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();
ExecutorService pool3 = Executors.newFixedThreadPool(threads);
List<Callable<String>> gets = new ArrayList<>();
for (int t = 0; t < 8; t++) gets.add(() -> cache.computeIfAbsent("k", k -> {
created.incrementAndGet(); return "v"; }));
pool3.invokeAll(gets); pool3.shutdown();
System.out.println("values created: " + created.get());                    // the mapping
function ran once, even with 8 racing callers
```

▶ Output

```
{hits=400000}
racy result <= expected: true
values created: 1
```

⚠️ Interview trap

`computeIfAbsent` on a `ConcurrentHashMap` blocks other updates to the same bucket while your function runs. If it makes a slow call (database, HTTP), you've serialised the whole bucket; load outside the lock and use `putIfAbsent` instead.

Q550 CL-071 • Advanced

What are weakly consistent iterators, and why is ConcurrentHashMap.size() only an estimate?

⚡ Quick answer

Concurrent collections' iterators never throw `ConcurrentModificationException`. A **weakly consistent** iterator reflects the state at (or some time after) its creation: it returns each element at most once, returns all elements that were present at creation and never removed, and **may or may not** reflect later changes. Likewise `size()` / `isEmpty()` are computed from counters while writers run, so they are estimates, not a point-in-time snapshot.

Iterator type	Examples	On concurrent modification
Fail-fast	`ArrayList`, `HashMap`, `TreeMap`	Best-effort `ConcurrentModificationException`

© TechTribers. For personal use only. Do not redistribute.

Snapshot	`CopyOnWriteArrayList`	Iterates the array as it was at creation
Weakly consistent	`ConcurrentHashMap`, `ConcurrentSkipListMap`, concurrent queues	No exception; may or may not see updates

Consequence for code: when you need an exact count or an atomic "read everything", a concurrent map can't give you one without external locking. Use `mappingCount()` for approximate size, an `AtomicLong` for an exact counter you maintain yourself, or take a copy (`new HashMap<>(chm)`) and accept that it's only weakly consistent too.

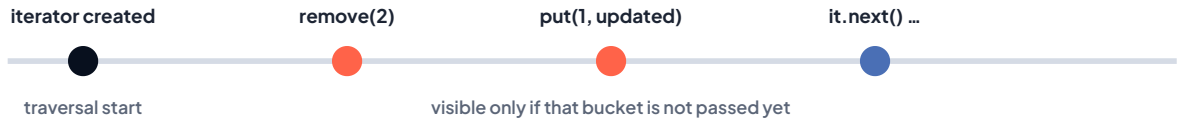
Java

```
ConcurrentHashMap<Integer, String> m = new ConcurrentHashMap<>();
for (int i = 1; i <= 3; i++) m.put(i, "v" + i);
for (Integer k : m.keySet()) { if (k == 2) m.remove(k); else m.put(k, "updated"); }       //
structural changes while iterating: no exception
System.out.println(new TreeMap<>(m));
Map<Integer, String> plain = new HashMap<>(Map.of(1, "a", 2, "b", 3, "c"));
try { for (Integer k : plain.keySet()) plain.put(k + 10, "x"); }
catch (ConcurrentModificationException e) { System.out.println("HashMap:
ConcurrentModificationException"); }
System.out.println(m.mappingCount() + " " + m.isEmpty());
```

▶ Output

```
{1=updated, 3=updated}
HashMap: ConcurrentModificationException
2 false
```

Weakly consistent iterator: never throws CME, may or may not see later changes



Guarantees: each element is returned at most once, and elements present at creation and never removed are all returned.

Same idea for ConcurrentSkipListMap and the queues. size() / isEmpty() are estimates while other threads write.

> 💡 **Pro tip**
>
> This is the "fail-safe vs fail-fast" interview question answered properly: "fail-safe" isn't an official term; the real categories are fail-fast, snapshot and weakly consistent.

© TechTribers. For personal use only. Do not redistribute.

Q551 CL-072 • Advanced

What are ConcurrentHashMap's bulk operations, and how do LongAdder and newKeySet help?

⚡ **Quick answer**

`ConcurrentHashMap` has parallel bulk methods: `forEach`, `search`, `reduce` (plus `reduceValues`, `reduceKeys` ...) take a **parallelism threshold** (1 = split as much as possible, `Long.MAX_VALUE` = sequential). For hot counters, store `LongAdder` values: they spread increments over striped cells and scale far better than `AtomicLong` / `merge` under contention. `ConcurrentHashMap.newKeySet()` is the standard concurrent `Set`.

The bulk methods run on the common `ForkJoinPool` and, like all weakly consistent operations, may see concurrent updates; the reducer must be associative and free of side effects. `LongAdder.sum()` is also not an atomic snapshot while writers run, but is exact once they stop. `computeIfAbsent(k, x -> new LongAdder()).increment()` is the usual per-key counter pattern (a metrics registry).

Java

```
ConcurrentHashMap<String, Integer> sales = new ConcurrentHashMap<>(Map.of("asha", 120, "ravi", 80,
"meena", 200, "kiran", 50));
long threshold = 1;                                                    // parallelism
threshold
int totalSales = sales.reduceValues(threshold, Integer::sum);
String firstBig = sales.search(threshold, (k, v) -> v > 150 ? k : null);                    //
first non-null result
String smallest = sales.reduce(threshold, (k, v) -> k + ":" + v, (a, b) -> a.compareTo(b) < 0 ? a :
b);   // smallest "key:value" string
System.out.println(totalSales + " " + firstBig + " " + smallest);

ConcurrentHashMap<String, LongAdder> hits = new ConcurrentHashMap<>();
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Callable<Void>> jobs = new ArrayList<>();
for (int t = 0; t < 4; t++) jobs.add(() -> { for (int i = 0; i < 50_000; i++)
hits.computeIfAbsent("page", k -> new LongAdder()).increment(); return null; });
pool.invokeAll(jobs); pool.shutdown();
System.out.println(hits.get("page").sum());                            // LongAdder scales
better than AtomicLong under contention
Set<String> concurrentSet = ConcurrentHashMap.newKeySet();
concurrentSet.addAll(List.of("a", "b", "a"));
System.out.println(concurrentSet.size());
```

▶ **Output**

```
450 meena asha:120
200000
2
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> For a single hot counter use `LongAdder` directly; for many counters keyed by name use the map-of-`LongAdder` pattern above.

📸 **Q552** `CL-073` • Intermediate

How do CopyOnWriteArrayList and CopyOnWriteArraySet work, and when should you use them?

> ⚡ **Quick answer**
> Every mutation (`add` , `set` , `remove`) takes a lock, **copies the whole backing array**, applies the change and publishes the new array. Readers never lock, and an iterator keeps the **array as it was when it was created**, so it never throws `ConcurrentModificationException` and doesn't see later changes. That makes it ideal for **read-mostly, small** collections such as listener lists, and poor for write-heavy or large ones.

Costs: each write is O(n) time and allocates O(n) memory, so 10,000 elements with frequent writes will thrash the GC. Benefits: zero-contention reads, and iteration is safe without any locking, perfect for the *observer pattern* (notifying listeners while others register or unregister).

`CopyOnWriteArraySet` is a thin wrapper that uses a `CopyOnWriteArrayList` and `addIfAbsent` , so `contains` is O(n): only use it for small sets.

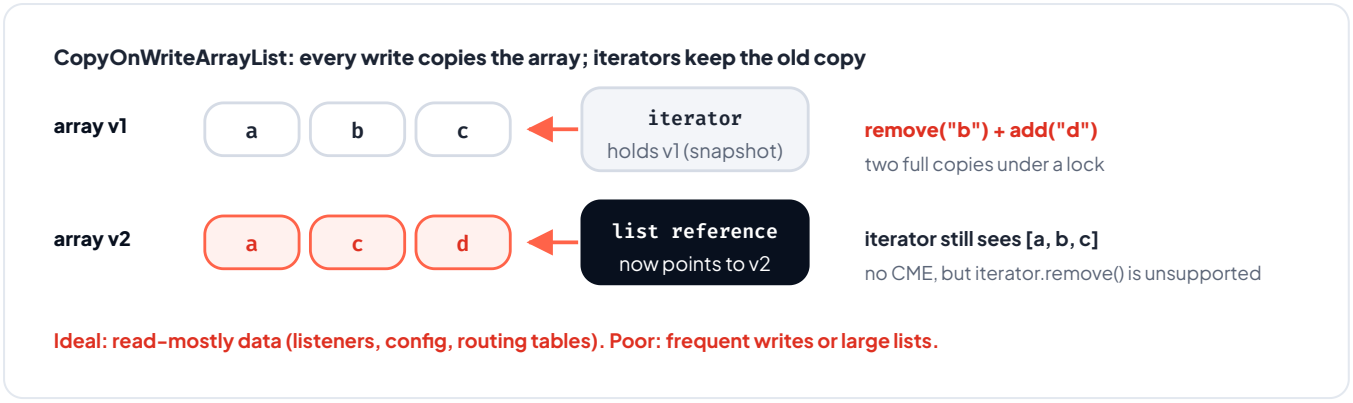
Java

```
CopyOnWriteArrayList<String> cow = new CopyOnWriteArrayList<>(List.of("a", "b", "c"));
for (String s : cow) {
    if (s.equals("a")) { cow.add("d"); cow.remove("b"); }                    // modify while
iterating: allowed
    System.out.print(s + " ");                                              // the iterator walks
the original snapshot
}
System.out.println();
System.out.println(cow);                                                    // the list itself has
changed
Iterator<String> it = cow.iterator();
try { it.next(); it.remove(); } catch (UnsupportedOperationException e) {
System.out.println("snapshot iterators can't remove"); }
CopyOnWriteArraySet<Integer> set = new CopyOnWriteArraySet<>(List.of(1, 2));
set.add(2); set.add(3);
System.out.println(set + " " + cow.addIfAbsent("c") + " " + cow.addIfAbsent("z"));
```

▶ **Output**

```
a b c
[a, c, d]
snapshot iterators can't remove
[1, 2, 3] false true
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

The snapshot means a loop can process an element that another thread has already removed (stale view), and `iterator.remove()` is not supported. If either is unacceptable, use a different structure.

Q553 CL-074 • Intermediate

How do Collections.synchronizedList and synchronizedMap work, and what do they not protect?

⚡ **Quick answer**

`Collections.synchronizedX(c)` returns a wrapper whose **every method locks on a single mutex** (the wrapper itself) before delegating. Single calls become atomic, but **iteration** and **compound actions** (check-then-act, `size()` then `get(i)`) are still races unless you hold the same lock: `synchronized (list) { for (...) ... }`. Contention is high because all operations share one lock.

When to use them: you already have a `HashMap` / `ArrayList` / `TreeMap` and need *basic* thread safety with the exact behaviour of that class (a **sorted** synchronized map, a synchronized `LinkedHashMap` LRU). When not to: new code that needs concurrency should use `ConcurrentHashMap`, `CopyOnWriteArrayList` or a `BlockingQueue`, which scale much better.

The wrapper holds the lock for the whole of `addAll` and `containsAll`, but `forEach`, `stream()`, `iterator()` and `spliterator()` on `synchronizedList` are *not* locked by the wrapper for the duration of the traversal: you must synchronise manually, exactly as shown.

Java

```
List<Integer> sync = Collections.synchronizedList(new ArrayList<>(List.of(1, 2, 3)));
sync.add(4);                                    // each single call is atomic
synchronized (sync) {                           // iteration and compound actions need the SAME
lock
    int sum = 0;
    for (int x : sync) sum += x;
    if (!sync.contains(5)) sync.add(5);         // check-then-act is only safe inside the block
    System.out.println(sum + " " + sync);
}
Map<String, Integer> smap = Collections.synchronizedMap(new TreeMap<>(Map.of("b", 2, "a", 1)));
synchronized (smap) { for (Map.Entry<String, Integer> e : smap.entrySet()) System.out.print(e + "
```


© TechTribers. For personal use only. Do not redistribute.

```
"); }
System.out.println();
System.out.println(sync.getClass().getSimpleName() + " " + smap.getClass().getSimpleName());
```

▶ Output

```
10 [1, 2, 3, 4, 5]
a=1 b=2
SynchronizedRandomAccessList SynchronizedMap
```

⚠️ **Interview trap**

`for (x : syncList)` without `synchronized (syncList)` can throw `ConcurrentModificationException` or skip elements under concurrent writes. The wrapper makes methods safe, not iteration.

Q554 CL-075 • Intermediate

What are the BlockingQueue implementations, and how do they differ?

⚡ **Quick answer**

A `BlockingQueue` adds operations that **wait**: `put` blocks while full, `take` blocks while empty, and `offer(e, timeout, unit)` / `poll(timeout, unit)` wait up to a limit. Implementations: `ArrayBlockingQueue` (bounded, array), `LinkedBlockingQueue` (optionally bounded, separate head/tail locks), `PriorityBlockingQueue` (unbounded, ordered), `SynchronousQueue` (zero capacity hand-off), `DelayQueue` (elements become available after a delay), `LinkedTransferQueue` (hand-off plus buffering).

Queue	Bounded	Order	Typical use
`ArrayBlockingQueue`	Yes (fixed)	FIFO	Back-pressure between stages, one lock
`LinkedBlockingQueue`	Optional	FIFO	Thread-pool work queue (higher throughput)
`PriorityBlockingQueue`	No	Priority	Prioritised tasks
`SynchronousQueue`	0 slots	n/a	`Executors.newCachedThreadPool` hand-off
`DelayQueue`	No	By expiry	Scheduling, cache expiry
`LinkedTransferQueue`	No	FIFO	`transfer()` waits for a consumer

An unbounded `LinkedBlockingQueue` in a thread pool is a classic outage cause: if producers outpace consumers the queue grows until the heap is exhausted. Prefer a bounded queue with a rejection policy.

Java

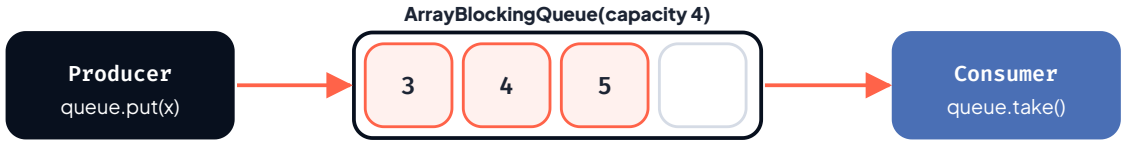
```
BlockingQueue<String> abq = new ArrayBlockingQueue<>(1);
System.out.println(abq.offer("a") + " " + abq.offer("b") + " " + abq.offer("b", 50,
TimeUnit.MILLISECONDS) + " " + abq.remainingCapacity());   // bounded: offer fails, the timed offer
waits then fails
BlockingQueue<String> lbq = new LinkedBlockingQueue<>();                    // unbounded
```


© TechTribers. For personal use only. Do not redistribute.

```
unless you pass a capacity
System.out.println(lbq.offer("x") + " " + lbq.remainingCapacity());
BlockingQueue<String> sq = new SynchronousQueue<>();                    // no storage: a
pure hand-off point
System.out.println(sq.offer("x") + " " + sq.size() + " " + sq.poll());  // no consumer
waiting, so offer() fails
PriorityBlockingQueue<Integer> pbq = new PriorityBlockingQueue<>(List.of(5, 1, 3));
System.out.println(pbq.take() + " " + pbq.take() + " " + pbq.take());
System.out.println(abq.poll(50, TimeUnit.MILLISECONDS) + " " + abq.poll(50, TimeUnit.MILLISECONDS));
// second poll waits 50 ms, then returns null
```

▶ Output

```
true false false 0
true 2147483646
false 0 null
1 3 5
a null
```



queue full → put() BLOCKS the producer (offer() returns false; offer(x, t, unit) waits)

queue empty → take() BLOCKS the consumer (poll() returns null; poll(t, unit) waits)

Back-pressure for free: a bounded queue slows a fast producer to the consumer's speed. Stop with a poison pill or interrupt.

⚠️ **Interview trap**

`ArrayBlockingQueue.add()` on a full queue throws `IllegalStateException("Queue full")`; `offer()` returns `false`; `put()` blocks. Choose the one whose failure mode you can handle.

Q555 CL-076 • Intermediate

How do you implement producer–consumer with a BlockingQueue?

⚡ **Quick answer**

Create a **bounded** `BlockingQueue`, have producers call `put(item)` and consumers call `take()`; the queue provides the locking, waiting and back-pressure, so you need no `wait` / `notify`. To stop consumers cleanly, send a **poison pill** (a sentinel value) per consumer, or interrupt the threads.

With several consumers, put **one poison pill per consumer**. For several producers, count how many finished and send the pills after the last one. Alternative shutdown: `executor.shutdownNow()` interrupts blocked `take()` calls, which then throw `InterruptedException`.

Why the old `wait` / `notify` version is worse: the guarded-block logic (`while (queue.isEmpty()) wait();`) is easy to get wrong (lost wake-ups, `if` instead of `while`, missing `notifyAll`), whereas `BlockingQueue`

© TechTribers. For personal use only. Do not redistribute.

encapsulates all of it.

```java
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(2);              // small capacity: the producer must wait for the consumer
final int POISON = -1;                                                   // sentinel that tells the consumer to stop
List<Integer> consumed = Collections.synchronizedList(new ArrayList<>());
Thread producer = new Thread(() -> {
    try { for (int i = 1; i <= 5; i++) queue.put(i); queue.put(POISON); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
Thread consumer = new Thread(() -> {
    try { for (int v; (v = queue.take()) != POISON; ) consumed.add(v); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
producer.start(); consumer.start();
producer.join(); consumer.join();
System.out.println("consumed " + consumed + " total " +
consumed.stream().mapToInt(Integer::intValue).sum());
```

▶ Output

```
consumed [1, 2, 3, 4, 5] total 15
```

💡 Pro tip

Interview follow-up: "what if the consumer is slower?" With a bounded queue the producer blocks on `put` (back-pressure). With an unbounded queue, memory grows without limit.

🚦 Q556 CL-077 • Intermediate

What are ConcurrentLinkedQueue and ConcurrentLinkedDeque?

⚡ Quick answer

`ConcurrentLinkedQueue` is an **unbounded, lock-free, non-blocking** FIFO queue (the Michael-Scott algorithm using CAS on node links); `ConcurrentLinkedDeque` is the double-ended version. `offer` and `poll` never block, never throw for capacity, and scale well with many threads. Downsides: `size()` is **O(n)** (it walks the list), no blocking `take`, and no capacity limit.

Choose it when consumers **poll** (e.g., a work-stealing or event-processing loop that does something else when empty), and you don't want to block threads. If consumers should sleep until work arrives, use a `LinkedBlockingQueue`. Don't call `size()` in a hot path; use `isEmpty()` or track your own `AtomicInteger`. Iterators are weakly consistent.

```java
Queue<Integer> q = new ConcurrentLinkedQueue<>();
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Callable<Void>> producers = new ArrayList<>();
```


© TechTribers. For personal use only. Do not redistribute.

```
for (int t = 0; t < 4; t++) { final int base = t * 1000; producers.add(() -> { for (int i = 0; i < 1000; i++) q.offer(base + i); return null; }); }
pool.invokeAll(producers); pool.shutdown();
System.out.println(q.size() + " " + q.isEmpty());                    // size() walks the whole queue: O(n)
ConcurrentLinkedDeque<String> dq = new ConcurrentLinkedDeque<>(List.of("b"));
dq.addFirst("a"); dq.addLast("c");
System.out.println(dq + " " + dq.pollLast());
```

▶ Output

```
4000 false
[a, b, c] c
```

⚠️ Interview trap

Because `ConcurrentLinkedQueue` is unbounded and non-blocking, a fast producer can exhaust memory with no back-pressure. Bound it yourself (with a counter or semaphore) or use a bounded `BlockingQueue`.

Q557 CL-078 • Advanced

What are ConcurrentSkipListMap and ConcurrentSkipListSet?

⚡ Quick answer

They are the **sorted, concurrent** map and set: a **skip list** (layers of linked lists where higher layers skip over more nodes) gives O(log n) expected time for `get`, `put`, `remove` and navigation (`floorKey`, `ceilingKey`, `headMap`, ...), without a global lock. A `TreeMap` is not thread-safe, and wrapping it with `synchronizedMap` serialises all access. They use lock-free CAS operations and are weakly consistent like other concurrent collections.

Use it for a **shared leaderboard, time-ordered event index, or ordered cache** accessed by many threads. Trade-offs: higher memory per entry than `ConcurrentHashMap`, slower point lookups (O(log n) vs O(1)), `size()` is O(n), and null keys/values are not allowed. If you only need sorted *snapshots* occasionally, a `ConcurrentHashMap` plus sorting the copy is often simpler and faster.

Java

```
ConcurrentSkipListMap<Integer, String> map = new ConcurrentSkipListMap<>();
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Callable<Void>> jobs = new ArrayList<>();
for (int t = 0; t < 4; t++) { final int base = t; jobs.add(() -> { for (int i = base; i < 20; i += 4) map.put(i, "v" + i); return null; }); }
pool.invokeAll(jobs); pool.shutdown();
System.out.println(map.size() + " " + map.firstKey() + " " + map.lastKey());                    // always sorted, safe for concurrent writers
System.out.println(map.floorKey(7) + " " + map.ceilingKey(21) + " " + map.headMap(3) + " " + map.subMap(5, 8));
ConcurrentSkipListSet<String> set = new ConcurrentSkipListSet<>(List.of("pear", "apple", "fig"));
System.out.println(set + " " + set.pollFirst());
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
20 0 19
7 null {0=v0, 1=v1, 2=v2} {5=v5, 6=v6, 7=v7}
[apple, fig, pear] apple
```

💡 Pro tip

A skip list's randomised structure is why it parallelises well: an insert only touches a few nodes, whereas a red-black tree may rebalance a whole path and needs a global lock.

⏰ Q558 CL-079 • Advanced

What is DelayQueue, and where is it used?

⚡ Quick answer

`DelayQueue<E extends Delayed>` is an unbounded blocking queue whose elements become **available only after their delay has expired**. `take()` blocks until the head's `getDelay()` is ≤ 0; elements are ordered by remaining delay. Use it for **scheduling** (retries with backoff, timeouts), **expiring cache entries**, and delayed task execution; `ScheduledThreadPoolExecutor` uses a similar internal delayed queue.

Implementation notes: `compareTo` must agree with `getDelay` (order by expiry time), use `System.nanoTime` (monotonic) rather than wall-clock time, and `size()` counts expired and unexpired elements alike. Only the head is examined, so a long-delay element at the head blocks consumers until it expires, even if later-queued elements are ready (that can't happen when `compareTo` orders by expiry).

Java

```
class Job implements Delayed {
    final String name; private final long readyAtNanos;
    Job(String name, long delayMs) { this.name = name; this.readyAtNanos = System.nanoTime() +
TimeUnit.MILLISECONDS.toNanos(delayMs); }
    @Override public long getDelay(TimeUnit unit) { return unit.convert(readyAtNanos -
System.nanoTime(), TimeUnit.NANOSECONDS); }
    @Override public int compareTo(Delayed other) { return Long.compare(readyAtNanos, ((Job)
other).readyAtNanos); }
}
DelayQueue<Job> queue = new DelayQueue<>();
queue.add(new Job("third", 300)); queue.add(new Job("first", 100)); queue.add(new Job("second",
200));
System.out.println(queue.poll() + " " + queue.size());                    // nothing is due yet:
poll() returns null, but all three are queued
StringBuilder order = new StringBuilder();
for (int i = 0; i < 3; i++) order.append(queue.take().name).append(' ');    // take() blocks until
the head's delay has expired
System.out.println(order);
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
null 3
first second third
```

⚠️ **Interview trap**

`peek()` / `poll()` return `null` while the head hasn't expired, but `size()` and iteration include unexpired elements. Don't use `isEmpty()` to mean "nothing ready".

Q559 CL-080

• Intermediate

How do you choose the right concurrent collection?

⚡ **Quick answer**

Start from the access pattern: shared **map** → `ConcurrentHashMap`; shared **sorted** map/set → `ConcurrentSkipListMap` / `Set`; **read-mostly list** → `CopyOnWriteArrayList`; **hand-off between threads** → a `BlockingQueue` (bounded for back-pressure); **lock-free polling** → `ConcurrentLinkedQueue` / `Deque`. Reach for `Collections.synchronizedX` only to wrap an existing collection with exact semantics.

Need	Use	Avoid
Counter map / cache	`ConcurrentHashMap` with `merge` / `computeIfAbsent`	`HashMap` + `synchronized` get/put sequences
Listeners, rarely-changing config	`CopyOnWriteArrayList`	Large or write-heavy lists
Worker pool hand-off	Bounded `ArrayBlockingQueue` / `LinkedBlockingQueue`	Unbounded queues without limits
Ordered concurrent index	`ConcurrentSkipListMap`	`synchronizedMap(new TreeMap<>())` under contention
Delayed work / expiry	`DelayQueue`, `ScheduledExecutorService`	Busy-wait with `sleep`
One thread owns the data	No sharing: confine to the thread, pass immutable messages	Locks everywhere

General principles: prefer **immutable snapshots** and **message passing** over shared mutable collections; keep critical sections tiny; never expose a synchronised collection's internal lock to callers; and always test with a start barrier and enough iterations, as the race demonstrations in this part do, because most concurrency bugs only show under real overlap.

© TechTribers. For personal use only. Do not redistribute.

Shared map, many readers and writers	`ConcurrentHashMap`
Sorted map or set, shared	`ConcurrentSkipListMap / Set`
Read-mostly list (listeners, config)	`CopyOnWriteArrayList / Set`
Hand-off between threads with back-pressure	`ArrayBlockingQueue / LinkedBlockingQueue`
Direct hand-off, no buffering	`SynchronousQueue`
Tasks by priority or by due time	`PriorityBlockingQueue / DelayQueue`
Lock-free queue or deque, no blocking	`ConcurrentLinkedQueue / Deque`
Existing HashMap/List you can't replace	`Collections.synchronizedX + manual lock`

💡 **Pro tip**

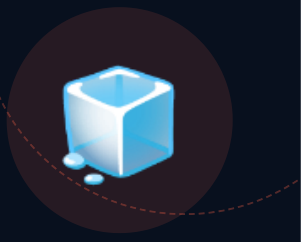
The strongest answer names the **trade-off**, not only the class: "ConcurrentHashMap trades exact size() and snapshot iteration for scalability."

Section 04

Immutable Collections & API Evolution

How the immutable factories behave, and what changed in the Collections API from Java 8 to Java 21.

Questions 560–561 · 2 questions



Q560 CL-081

• Intermediate

Why does the iteration order of Set.of and Map.of change between JVM runs?

⚡ **Quick answer**

The immutable `Set` / `Map` factories (Java 9) deliberately **randomise iteration order per JVM run**: a salt computed at class-initialisation time is mixed into the probe sequence. The `Set` / `Map` contracts promise no order, and the JDK team randomised it so programs can't accidentally depend on one. They also **reject null**, duplicates (`IllegalArgumentException`), and are immutable, space-efficient and fast to create.

The same program run four times in separate JVMs, printing `Set.of("a","b","c","d","e")` and `Map.of("x",1,"y",2,"z",3,"w",4)` directly (measured on JDK 25):

Run	`Set.of(...)`	`Map.of(...)`

© TechTribers. For personal use only. Do not redistribute.

1	[e, d, c, b, a]	{z=3, y=2, x=1, w=4}
2	[c, d, e, a, b]	{w=4, x=1, y=2, z=3}
3	[c, d, e, a, b]	{w=4, x=1, y=2, z=3}
4	[e, d, c, b, a]	{z=3, y=2, x=1, w=4}

`List.of` is different: order is **always** the argument order (a `List` is ordered). Why the randomisation matters: tests that compare `Set.of(...).toString()` to a fixed string pass sometimes and fail other times, and code that relies on a particular order breaks the day the JDK changes the hashing. If you need a stable order, use `new LinkedHashSet<>(List.of(...))` or a `TreeSet`.

Java

```
Set<String> s = Set.of("a", "b", "c", "d", "e");
System.out.println(new TreeSet<>(s));                                    // sort if you need a stable order
Map<String, Integer> m = Map.of("x", 1, "y", 2, "z", 3);
System.out.println(new TreeMap<>(m));
try { Set.of("a", "a"); } catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
try { Map.of("k", 1, "k", 2); } catch (IllegalArgumentException e) {
System.out.println(e.getMessage()); }
try { List.of("a").contains(null); } catch (NullPointerException e) {
System.out.println("contains(null) throws too"); }
Map<String, Integer> big = Map.ofEntries(Map.entry("a", 1), Map.entry("b", 2));          // more than 10 pairs: use ofEntries
Map<String, Integer> copy = Map.copyOf(new HashMap<>(big));
System.out.println(copy.equals(big) + " " + (Map.copyOf(big) == big));                   // copyOf returns the SAME instance when already immutable
```

▶ Output

```
[a, b, c, d, e]
{x=1, y=2, z=3}
duplicate element: a
duplicate key: k
contains(null) throws too
true true
```

⚠️ **Interview trap**

`Set.copyOf(list)` throws nothing for duplicates (it de-duplicates), but `Set.of(a, b)` with equal arguments throws. `List.of(...)` with a null element throws `NullPointerException`; `Arrays.asList(...)` allows it.

© TechTribers. For personal use only. Do not redistribute.

Q561 CL-082 • Intermediate

What changed in the Collections API between Java 8 and Java 21?

⚡ **Quick answer**

Java 8: default methods (`forEach` , `removeIf` , `replaceAll` , `Map.compute*` / `merge` , `Comparator` combinators) and streams. Java 9: `List.of` / `Set.of` / `Map.of` , `Map.entry` . Java 10: `copyOf` , `Collectors.toUnmodifiable*` . Java 16: `Stream.toList()` . Java 19: `HashMap.newHashMap` . Java 21: **sequenced collections** (`getFirst` , `getLast` , `addFirst` , `reversed()` , `SequencedMap`).

Release	Collections additions
8	`forEach` , `removeIf` , `replaceAll` , `Map.getOrDefault/putIfAbsent/compute/merge` , `List.sort` , `Spliterator` , streams, `Comparator.comparing/thenComparing`
9	`List.of` , `Set.of` , `Map.of` , `Map.ofEntries` , `Map.entry` , `Stream.takeWhile/dropWhile/ofNullable`
10	`List.copyOf` / `Set.copyOf` / `Map.copyOf` , `Collectors.toUnmodifiableList/Set/Map`
12	`Collectors.teeing`
16	`Stream.toList()` , `Stream.mapMulti`
19	`HashMap.newHashMap` , `HashSet.newHashSet` , `LinkedHashMap.newLinkedHashMap`
21	`SequencedCollection` / `SequencedSet` / `SequencedMap` , `List.reversed/getFirst/addFirst` , `Collections.unmodifiableSequenced*`
24	Stream Gatherers (`windowFixed` , `scan` , ...)

Not in the list: no new general-purpose collection classes were added; the JDK's strategy has been better factories, immutability and ordering interfaces rather than new data structures (value-class-based collections wait on Project Valhalla).

Java

```
List<Integer> a = List.of(1, 2, 3);                                    // Java 9: immutable factories
List<Integer> b = List.copyOf(new ArrayList<>(a));                     // Java 10: immutable copies (same instance if already immutable)
System.out.println((List.copyOf(a) == a) + " " + b);
System.out.println(Stream.of(3, 1, 2).sorted().toList());              // Java 16: Stream.toList() (unmodifiable)
System.out.println(Stream.of(1, 2, 3).collect(Collectors.toUnmodifiableSet()).size());      // Java 10
Map<String, Integer> m = HashMap.newHashMap(10); m.put("k", 1);        // Java 19: sized for N entries
List<String> names = new ArrayList<>(List.of("a", "b", "c"));
System.out.println(names.reversed() + " " + names.getFirst() + " " + names.getLast());     // Java 21: SequencedCollection
names.addFirst("z"); names.removeLast();
System.out.println(names);
LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>(); lhm.put("x", 1); lhm.putFirst("w", 0);
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(lhm + " " + lhm.firstEntry() + " " + lhm.sequencedKeySet().reversed());
System.out.println(Map.entry("k", 1) + " " + Objects.requireNonNullElse(null, "default"));
```

▶ Output

```
true [1, 2, 3]
[1, 2, 3]
3
[c, b, a] a c
[z, a, b]
{w=0, x=1} w=0 [x, w]
k=1 default
```

💡 **Pro tip**

In an interview, group by theme: *convenience* (factories, `copyOf`), *functional* (default methods, collectors, streams), *ordering* (sequenced collections) and *sizing* (`newHashMap`).

Section 05

Collectors, Streams & Collections

Grouping, partitioning, toMap pitfalls, flattening, inverting maps, top-N per group, counting, and parallel streams on collections.

Questions 562–569 · 8 questions



🗂️ **Q562** CL-083 • Intermediate

How does Collectors.groupingBy work, with downstream collectors and a map factory?

⚡ **Quick answer**

`groupingBy(classifier)` builds a `Map<K, List<T>>` where the classifier function gives the **key**. Add a **downstream collector** to compute the value per group (`counting()`, `mapping(f, toList())`, `summingInt`, `averagingDouble`, `maxBy` ...) and a **map factory** (`TreeMap::new`, `LinkedHashMap::new`) to control the map type and key order.

Details worth knowing:

- Without a map factory you get a `HashMap`, so the key order is unspecified. A `null` classifier result throws `NullPointerException`.
- `groupingBy` never creates an empty group; a key exists only if at least one element mapped to it.
- `maxBy` returns `Optional`, because a group can't be empty here but the collector is generic; use `collectingAndThen(maxBy(...), Optional::get)` to unwrap.
- `groupingByConcurrent` builds a `ConcurrentMap` for parallel streams (unordered).

© TechTribers. For personal use only. Do not redistribute.

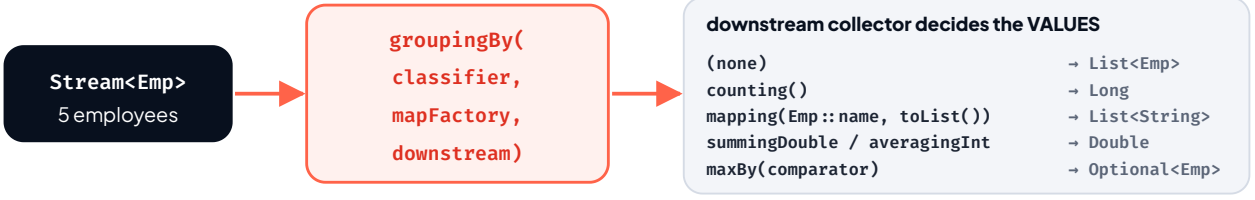
- You can nest: `groupingBy(dept, groupingBy(city, counting()))` gives a two-level `Map<String, Map<String, Long>>`.

Java

```
static void show(Object o) { System.out.println(o); }                    // avoids a println
overload-inference quirk with generic collect() results
record Emp(String name, String dept, double salary, int age) { }
List<Emp> staff = List.of(new Emp("Asha", "ENG", 120, 30), new Emp("Ravi", "ENG", 100, 41), new
Emp("Meena", "OPS", 80, 28),
        new Emp("Kiran", "OPS", 90, 35), new Emp("Dev", "HR", 70, 26));
System.out.println(staff.stream().collect(Collectors.groupingBy(Emp::dept)).keySet().stream().sorted
().toList());        // Map<String, List<Emp>>
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new, Collectors.counting())));
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.mapping(Emp::name, Collectors.toList()))));
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.summingDouble(Emp::salary))));
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.averagingInt(Emp::age))));
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.maxBy(Comparator.comparingDouble(Emp::salary)))).values().stream().map(o ->
o.get().name()).toList());
show(staff.stream().collect(Collectors.groupingBy(e -> e.age() / 10 * 10, TreeMap::new,
Collectors.counting())));      // group by a computed key (age decade)
show(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.collectingAndThen(Collectors.toList(), List::size))));
```

▶ Output

```
[ENG, HR, OPS]
{ENG=2, HR=1, OPS=2}
{ENG=[Asha, Ravi], HR=[Dev], OPS=[Meena, Kiran]}
{ENG=220.0, HR=70.0, OPS=170.0}
{ENG=35.5, HR=26.0, OPS=31.5}
[Asha, Dev, Kiran]
{20=2, 30=2, 40=1}
{ENG=2, HR=1, OPS=2}
```



Result: Map<K, R>. The classifier gives the KEYS; HashMap by default, pass TreeMap::new (or LinkedHashMap::new) for order. partitioningBy is groupingBy with a boolean classifier: both keys (false, true) are always present.

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `groupingBy` + `counting()` is the one-liner for a frequency table; `Collectors.toMap(k, v, merge, TreeMap::new)` is its cousin for "one value per key" (next questions).

Q563 CL-084 • Basic

What is the difference between groupingBy and partitioningBy?

> ⚡ **Quick answer**
> `partitioningBy(predicate)` splits elements into exactly **two groups**, keyed by `false` and `true`, and **both keys are always present** even when a group is empty. `groupingBy(classifier)` can create any number of groups and includes **only groups that have elements**. Both accept a downstream collector.

Because both keys always exist, `partitioningBy` results are safe to read with `parts.get(true)` without a null check, which is the main practical advantage. The returned map is a special immutable-ish two-entry map (`Partition`): `put` throws, and it prints in `false, true` order.

Java

```
List<Integer> nums = IntStream.rangeClosed(1, 8).boxed().toList();
System.out.println(nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0)));
System.out.println(nums.stream().filter(n -> n > 100).collect(Collectors.partitioningBy(n -> n % 2
== 0)));      // BOTH keys present, even when empty
System.out.println(nums.stream().filter(n -> n > 100).collect(Collectors.groupingBy(n -> n % 2 ==
0)));          // groupingBy: no entry for an empty group
System.out.println(nums.stream().collect(Collectors.partitioningBy(n -> n > 5,
Collectors.summingInt(n -> n))));
Map<Boolean, List<Integer>> parts = nums.stream().collect(Collectors.partitioningBy(n -> n < 3));
System.out.println(parts.get(true) + " " + parts.get(false).size());
```

▶ **Output**

```
{false=[1, 3, 5, 7], true=[2, 4, 6, 8]}
{false=[], true=[]}
{}
{false=15, true=21}
[1, 2] 6
```

> 💡 **Pro tip**
> Use `partitioningBy` for yes/no splits (valid vs invalid records, pass vs fail), and `groupingBy` for categories.

© TechTribers. For personal use only. Do not redistribute.

Q564 CL-085 • Intermediate

What are the pitfalls of Collectors.toMap?

⚡ Quick answer

`toMap(keyFn, valueFn)` throws `IllegalStateException` **on duplicate keys** unless you pass a **merge function**, throws `NullPointerException` **for null values**, and returns a `HashMap` (no order) unless you pass a map supplier. Choose `toMap` when each key should appear once and `groupingBy` when a key can have many elements.

Which collector, quickly:

- One value per key, keys known to be unique → `toMap(k, v)`.
- Possible duplicates, keep first/last/combine → `toMap(k, v, (a, b) -> a)` (or `b`, or concatenate).
- Many values per key → `groupingBy(k)` or `groupingBy(k, mapping(v, toList()))`.
- Need ordering → add `LinkedHashMap::new` (encounter order) or `TreeMap::new` (key order) as the fourth argument.
- Null values possible → `toMap` fails; fill the map with a loop and `HashMap.put`, or wrap values in `Optional`.

Java

```
static void show(Object o) { System.out.println(o); }                    // avoids a println
overload-inference quirk with generic collect() results
record Emp(int id, String dept, String name) { }
List<Emp> staff = List.of(new Emp(1, "ENG", "Asha"), new Emp(2, "ENG", "Ravi"), new Emp(3, "OPS",
"Meena"));
System.out.println(staff.stream().collect(Collectors.toMap(Emp::id, Emp::name)));
try { staff.stream().collect(Collectors.toMap(Emp::dept, Emp::name)); }
catch (IllegalStateException e) { System.out.println(e.getMessage()); }
// duplicate key "ENG"
show(staff.stream().collect(Collectors.toMap(Emp::dept, Emp::name, (a, b) -> a + "+" + b,
TreeMap::new)));      // merge function + ordered map
try { Stream.of("a", "b").collect(Collectors.toMap(s -> s, s -> (String) null)); }
catch (NullPointerException e) { System.out.println("null values: NullPointerException"); }
System.out.println(staff.stream().collect(Collectors.toMap(Emp::id,
Function.identity()))).getClass().getSimpleName());       // HashMap: no order guarantee
System.out.println(staff.stream().collect(Collectors.toUnmodifiableMap(Emp::id, Emp::name)).size());
```

▶ Output

```
{1=Asha, 2=Ravi, 3=Meena}
Duplicate key ENG (attempted merging values Asha and Ravi)
{ENG=Asha+Ravi, OPS=Meena}
null values: NullPointerException
HashMap
3
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

The `NullPointerException` from `toMap` with a null value comes from `HashMap.merge`, so the stack trace doesn't mention your lambda. It is a well-known annoyance (JDK-8148463).

Q565 CL-086 • Basic

How do you flatten nested collections with flatMap?

⚡ **Quick answer**

`flatMap(f)` maps each element to a **stream** and concatenates the results into one stream: `List<List<T>>` → `Stream<T>` with `flatMap(List::stream)`. Use it for nested lists, `Map<K, List<V>>` values, splitting strings into words, and 2-D arrays (`flatMapToInt`). To keep the outer key, build the combined value inside the lambda.

`map` produces one output per input (`Stream<Stream<T>>` if the function returns a stream), `flatMap` unwraps one level. Empty inner collections simply contribute nothing (as `meena` does above), and an inner `null` stream is treated as empty. Java 16's `mapMulti` is the allocation-free alternative when each element produces few results.

Java

```
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3), List.of(), List.of(4, 5));
System.out.println(nested.stream().flatMap(List::stream).toList());
Map<String, List<String>> skills = new TreeMap<>(Map.of("asha", List.of("java", "sql"), "ravi",
List.of("java", "go"), "meena", List.of()));
System.out.println(skills.values().stream().flatMap(List::stream).distinct().sorted().toList());
System.out.println(skills.entrySet().stream().flatMap(e -> e.getValue().stream().map(s -> e.getKey()
+ ":" + s)).toList());      // keep the key while flattening
System.out.println(Stream.of("a,b", "c").flatMap(s -> Arrays.stream(s.split(","))).toList());
List<String> words = List.of("hi", "yo");
System.out.println(words.stream().flatMap(w -> w.chars().mapToObj(c -> (char) c)).toList());
int[][] grid = {{1, 2}, {3, 4}};
System.out.println(Arrays.stream(grid).flatMapToInt(Arrays::stream).sum());
```

▶ **Output**

```
[1, 2, 3, 4, 5]
[go, java, sql]
[asha:java, asha:sql, ravi:java, ravi:go]
[a, b, c]
[h, i, y, o]
10
```

💡 **Pro tip**

Think "flat" = the inner container disappears: `Optional<Optional<T>>` → `Optional<T>`, `Stream<Stream<T>>` → `Stream<T>`.

© TechTribers. For personal use only. Do not redistribute.

Q566 CL-087

• Intermediate

How do you invert a Map (swap keys and values), including duplicate values?

⚡ **Quick answer**

If values are **unique**, stream the entries and `toMap(Entry::getValue, Entry::getKey)`. If values can repeat (so inversion would lose keys), group instead: `groupingBy(Entry::getValue, mapping(Entry::getKey, toList()))` gives `Map<V, List<K>>`. A plain `toMap` on duplicate values throws `IllegalStateException`.

A true bidirectional map (**BiMap**) enforces unique values on insert and keeps both directions in sync (Guava's `BiMap`, or the small implementation in the multimap/bimap question below). For one-off lookups by value, `map.entrySet().stream().filter(e -> value.equals(e.getValue()))` is O(n); invert once if you will search by value repeatedly.

Java

```
Map<String, String> capital = new TreeMap<>(Map.of("India", "Delhi", "Japan", "Tokyo", "France",
"Paris"));
Map<String, String> byCapital = capital.entrySet().stream()
        .collect(Collectors.toMap(Map.Entry::getValue, Map.Entry::getKey, (a, b) -> a,
TreeMap::new));       // values must be unique
System.out.println(byCapital);
Map<String, Integer> grades = new TreeMap<>(Map.of("asha", 90, "ravi", 80, "meena", 90, "kiran", 80,
"dev", 70));
Map<Integer, List<String>> byGrade = grades.entrySet().stream().collect(
        Collectors.groupingBy(Map.Entry::getValue, TreeMap::new,
Collectors.mapping(Map.Entry::getKey, Collectors.toList())));
System.out.println(byGrade);                                                    // duplicates:
invert to Map<V, List<K>>
Map<String, Integer> back = new TreeMap<>();
byGrade.forEach((g, names) -> names.forEach(n -> back.put(n, g)));
System.out.println(back.equals(grades));                                        // the round
trip restores the original
```

▶ **Output**

```
{Delhi=India, Paris=France, Tokyo=Japan}
{70=[dev], 80=[kiran, ravi], 90=[asha, meena]}
true
```

⚠️ **Interview trap**

`toMap(Entry::getValue, Entry::getKey)` on `{asha=90, meena=90}` throws `IllegalStateException: Duplicate key 90`. Decide up front whether duplicates are possible.

© TechTribers. For personal use only. Do not redistribute.

🏅 Q567 CL-088 • Advanced

How do you get the top N elements per group?

⚡ Quick answer

Group with `groupingBy`, then use the `collectingAndThen` downstream collector to sort each group's list and take `limit(N)`: `groupingBy(dept, collectingAndThen(toList(), l -> l.stream().sorted(cmp).limit(2).toList()))`. For only the top one** per group, `maxBy(comparator)` is simpler and doesn't build a list. For one overall top N, sort the whole stream and `limit(N)`.

Complexity: sorting each group costs O(g log g). For very large groups where N is small, keep a size-N `PriorityQueue` per group instead (O(g log N), as in the top-K question in Part 5). In SQL this is `ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) <= N`, a frequent follow-up ("how would you do it in the database?").

Java

```
record Emp(String name, String dept, int salary) { }
List<Emp> staff = List.of(new Emp("Asha", "ENG", 120), new Emp("Ravi", "ENG", 100), new Emp("Neha",
"ENG", 140),
        new Emp("Meena", "OPS", 80), new Emp("Kiran", "OPS", 90), new Emp("Dev", "HR", 70));
Map<String, List<String>> top2 = staff.stream().collect(Collectors.groupingBy(Emp::dept,
TreeMap::new,
        Collectors.collectingAndThen(Collectors.toList(),
                list ->
list.stream().sorted(Comparator.comparingInt(Emp::salary).reversed()).limit(2).map(Emp::name).toList
())));
System.out.println(top2);
System.out.println(staff.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.maxBy(Comparator.comparingInt(Emp::salary)))))
        .entrySet().stream().map(e -> e.getKey() + "=" + e.getValue().get().name()).toList());
System.out.println(staff.stream().sorted(Comparator.comparingInt(Emp::salary).reversed()).limit(3).m
ap(Emp::name).toList());       // overall top 3
```

▶ Output

```
{ENG=[Neha, Asha], HR=[Dev], OPS=[Kiran, Meena]}
[ENG=Neha, HR=Dev, OPS=Kiran]
[Neha, Asha, Ravi]
```

💡 Pro tip

Say the stream version, then mention the SQL window-function equivalent. It shows you know where the work belongs for large data.

© TechTribers. For personal use only. Do not redistribute.

Q568 CL-089

• Basic

How do you count occurrences of each element and find the most frequent one?

⚡ Quick answer

Build a frequency map with `groupingBy(identity(), counting())` or the classic loop `map.merge(x, 1, Integer::sum)`. The most frequent entry is `max(Map.Entry.comparingByValue())`; for top-K or tie-breaking, sort by value descending then by key. Elements occurring more than once come from `filter(!seen.add(x))` or by filtering counts `> 1`.

`max` on ties returns whichever entry the comparator treats as largest last-seen; add `thenComparing(key)` for a deterministic answer. The **Boyer–Moore voting** algorithm finds a majority element (more than n/2) in O(1) space (covered in the coding part). For huge streams that don't fit in memory, use a count-min sketch or a bounded heavy-hitters algorithm.

Java

```
List<String> words = List.of("java", "go", "java", "rust", "go", "java");
Map<String, Long> counts = words.stream().collect(Collectors.groupingBy(Function.identity(),
TreeMap::new, Collectors.counting()));
System.out.println(counts);
System.out.println(counts.entrySet().stream().max(Map.Entry.comparingByValue()).get());
// the most frequent element
System.out.println(counts.entrySet().stream().sorted(Map.Entry.<String,
Long>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()))
        .limit(2).map(Map.Entry::getKey).toList());
Set<String> seen = new HashSet<>();
System.out.println(words.stream().filter(w -> !seen.add(w)).distinct().toList());
// elements that occur more than once
System.out.println(counts.entrySet().stream().filter(e -> e.getValue() ==
1).map(Map.Entry::getKey).toList());           // elements that occur exactly once
Map<String, Integer> classic = new TreeMap<>();
for (String w : words) classic.merge(w, 1, Integer::sum);
// the pre-stream loop, still the fastest
System.out.println(classic);
```

▶ Output

```
{go=2, java=3, rust=1}
java=3
[java, go]
[java, go]
[rust]
{go=2, java=3, rust=1}
```

💡 Pro tip

`counting()` returns `Long`, while `merge(x, 1, Integer::sum)` produces `Integer`. Mixing them up is a frequent compile error when assigning the map.

© TechTribers. For personal use only. Do not redistribute.

⚡ Q569 CL-090 • Advanced

What pitfalls do parallel streams have on collections?

> ⚡ Quick answer
> Parallel streams split the source, so they help only for **large, cheaply splittable sources** (`ArrayList` , arrays, ranges, `HashMap` more or less) with **CPU-heavy, independent, stateless** operations. They hurt with `LinkedList` / `Stream.iterate` (poor splitting), tiny sizes (overhead), blocking I/O (starves the shared common pool), and anything with **shared mutable state** or **non-associative** reductions.

Reading the first line: the sequential result is -36, but the parallel one differs because the combiner applies `a - b` between partial results whose grouping depends on how the stream was split. (The program pins the common pool to 4 threads so the split, and therefore the printed number, is the same on every machine; on other core counts the parallel value can differ, which is exactly the bug.)

Rules of thumb:

- Reductions must be **associative**, with an identity element; functions **stateless** and **non-interfering**.
- Never `add` to a shared `ArrayList` or `HashMap` from `forEach` on a parallel stream. Use `collect` , `toConcurrentMap` or `groupingByConcurrent` .
- All parallel streams share the **common ForkJoinPool** (see the parallel-streams question in Part 4); one slow blocking task delays every other parallel stream in the JVM.
- Measure first. For small collections or cheap per-element work, sequential is faster.

(I couldn't measure parallel speed-ups for this book: the sandbox has a single CPU core, where parallel streams only add overhead. Benchmark on your target hardware with JMH.)

Java

```
List<Integer> nums = IntStream.rangeClosed(1, 8).boxed().toList();
int seq = nums.stream().reduce(0, (a, b) -> a - b);
int par = nums.parallelStream().reduce(0, (a, b) -> a - b);            // subtraction is NOT
associative: the result depends on how the stream is split
System.out.println(seq + " " + par + " " + (seq == par));
System.out.println(nums.parallelStream().map(n -> n * 2).collect(Collectors.toList()));
// collect keeps encounter order
System.out.println(nums.parallelStream().reduce(0, Integer::sum) + " " +
nums.parallelStream().mapToInt(i -> i).sum());    // associative operations are safe
List<Integer> sink = new ArrayList<>();
nums.parallelStream().forEachOrdered(sink::add);                       // forEachOrdered preserves
order (and gives up parallelism)
System.out.println(sink);
System.out.println(nums.parallelStream().collect(Collectors.groupingByConcurrent(n -> n % 2 == 0,
Collectors.counting())));
System.out.println(nums.parallelStream().filter(n -> n > 4).findFirst().get() + " " +
nums.parallelStream().unordered().filter(n -> n > 4).findAny().isPresent());
```

▶ Output

```
-36 0 false
[2, 4, 6, 8, 10, 12, 14, 16]
36 36
[1, 2, 3, 4, 5, 6, 7, 8]
```


© TechTribers. For personal use only. Do not redistribute.

```
{false=4, true=4}
5 true
```

⚠️ **Interview trap**

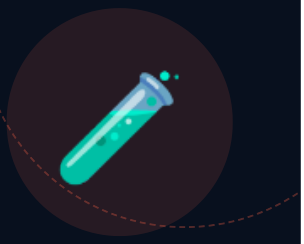
`findFirst()` and `forEachOrdered` keep order and reduce parallel benefit; `findAny()` and `unordered()` allow more parallelism but return arbitrary results. Pick the one that matches the requirement.

Section 06

Practical Map & Cache Problems

The coding-round problems that use maps and queues: word counts, your own HashMap, multimaps, a TTL cache, a rate limiter, merging maps and a time-based store.

Questions 570–576 · 7 questions



📝 **Q570** CL-091 • Basic

Count word frequencies and return the top K words.

⚡ **Quick answer**

Count with `map.merge(word, 1, Integer::sum)` (O(n)), then pick the top K by sorting entries by count descending and word ascending (O(m log m) for m distinct words), or keep a **size-K min-heap** (O(m log K)). Normalise first (lower-case, strip punctuation) and define the tie-break explicitly.

Choose the heap when `m` is large and `K` is small (log K instead of log m per element). For streaming data that never ends, use a heavy-hitter algorithm (Misra-Gries, count-min sketch). Real text needs `toLowerCase`, splitting on `\\W+`, and often stop-word removal.

Java

```
static void show(Object o) { System.out.println(o); }                    // avoids a println
overload-inference quirk with generic collect() results
String text = "the quick brown fox jumps over the lazy dog the fox";
Map<String, Integer> freq = new HashMap<>();
for (String w : text.split("\\s+")) freq.merge(w, 1, Integer::sum);
System.out.println(new TreeMap<>(freq));
List<String> top2 = freq.entrySet().stream()
        .sorted(Map.Entry.<String,
Integer>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()))
        .limit(2).map(e -> e.getKey() + "=" + e.getValue()).toList();
System.out.println(top2);
PriorityQueue<Map.Entry<String, Integer>> heap = new PriorityQueue<>(           // min-heap of size
K: the weakest of the top K is at the head
        Map.Entry.<String,
```


© TechTribers. For personal use only. Do not redistribute.

```
Integer>comparingByValue().thenComparing(Map.Entry.comparingByKey(Comparator.reverseOrder())));
for (Map.Entry<String, Integer> e : freq.entrySet()) { heap.offer(e); if (heap.size() > 2)
heap.poll(); }
List<String> viaHeap = new ArrayList<>();
while (!heap.isEmpty()) viaHeap.add(0, heap.poll().getKey());
System.out.println(viaHeap);
show(Arrays.stream(text.split("\\s+")).collect(Collectors.groupingBy(String::length, TreeMap::new,
Collectors.counting())));
```

▶ Output

```
{brown=1, dog=1, fox=2, jumps=1, lazy=1, over=1, quick=1, the=3}
[the=3, fox=2]
[the, fox]
{3=6, 4=2, 5=3}
```

💡 **Pro tip**

Ask the interviewer about ties (alphabetical? first seen?) and case sensitivity before coding; both change the comparator.

Q571 CL-092 • Advanced

Implement your own HashMap (chaining and resizing).

⚡ **Quick answer**

Keep an array of buckets, each a singly-linked chain of nodes. `put` computes the index from the key's spread hash, walks the chain replacing an equal key, otherwise adds a node; `get` and `remove` walk the same chain. Resize (double the array and re-insert every node) when `size > 0.75 × capacity`. Handle the `null` key as hash 0 and compare keys with `Objects.equals`.

What interviewers look for: a power-of-two capacity with a mask index, `equals` plus `hashCode` (not `==`), updating an existing key instead of duplicating it, resize on load factor, correct `remove` of the head node, and the `null` key. Discuss extensions: tree bins for long chains, open addressing with linear probing as an alternative, thread safety (per-bucket locks), iteration (keys, values, entries) and fail-fast `modCount`.

Differences from the JDK: this version inserts at the **head** of a chain (the JDK appends at the tail since Java 8) and never converts chains to trees.

Java

```
class MyHashMap<K, V> {
    private static final class Node<K, V> {
        final K key; V val; Node<K, V> next;
        Node(K key, V val, Node<K, V> next) { this.key = key; this.val = val; this.next = next; }
    }
    private Node<K, V>[] table;
    private int size;
    @SuppressWarnings("unchecked") MyHashMap() { table = (Node<K, V>[]) new Node[4]; }

    private static int index(Object key, int n) { int h = key == null ? 0 : key.hashCode(); return
```

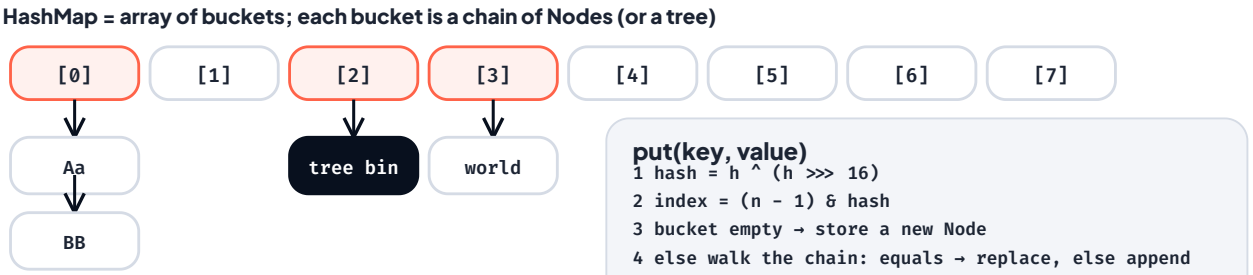

© TechTribers. For personal use only. Do not redistribute.

```
(h ^ (h >>> 16)) & (n - 1); }
    V put(K key, V val) {
        int i = index(key, table.length);
        for (Node<K, V> n = table[i]; n != null; n = n.next)
            if (Objects.equals(n.key, key)) { V old = n.val; n.val = val; return old; }
        table[i] = new Node<>(key, val, table[i]);                          // insert at the head of
the chain
        if (++size > table.length * 3 / 4) resize();
        return null;
    }
    V get(Object key) {
        for (Node<K, V> n = table[index(key, table.length)]; n != null; n = n.next) if
(Objects.equals(n.key, key)) return n.val;
        return null;
    }
    V remove(Object key) {
        int i = index(key, table.length);
        for (Node<K, V> n = table[i], prev = null; n != null; prev = n, n = n.next)
            if (Objects.equals(n.key, key)) { if (prev == null) table[i] = n.next; else prev.next =
n.next; size--; return n.val; }
        return null;
    }
    @SuppressWarnings("unchecked") private void resize() {
        Node<K, V>[] old = table;
        table = (Node<K, V>[]) new Node[old.length * 2];
        for (Node<K, V> head : old)
            for (Node<K, V> n = head; n != null; ) { Node<K, V> next = n.next; int i = index(n.key,
table.length); n.next = table[i]; table[i] = n; n = next; }
    }
    int size() { return size; }
    int capacity() { return table.length; }
}
MyHashMap<String, Integer> m = new MyHashMap<>();
for (String k : List.of("a", "b", "c", "d", "e", "f", "g")) m.put(k, k.charAt(0) - 'a');
System.out.println(m.size() + " " + m.capacity() + " " + m.get("e") + " " + m.get("zzz") + " " +
m.put("e", 99) + " " + m.get("e"));
System.out.println(m.remove("a") + " " + m.remove("a") + " " + m.size());
MyHashMap<Object, String> nulls = new MyHashMap<>();
nulls.put(null, "n");
System.out.println(nulls.get(null));
```

▶ Output

```
7 16 4 null 4 99
0 null 6
n
```


© TechTribers. For personal use only. Do not redistribute.



Node = (hash, key, value, next). get() repeats steps 1–2 and walks the bucket. A null key hashes to 0. Not thread-safe.

> 💡 **Pro tip**
> Write `put` / `get` first, then `resize`, then `remove`. Narrating the invariants ("every node lives at index(key, table.length)") shows you understand why resize must re-insert.

Q572 CL-093 • Intermediate

How do you implement a multimap, a counter (bag) and a bidirectional map with plain collections?

> ⚡ **Quick answer**
> Multimap: `Map<K, List<V>>` (or `Set<V>`) populated with `computeIfAbsent(k, x -> new ArrayList<>()).add(v)`. Bag/counter: `Map<T, Integer>` with `merge(t, 1, Integer::sum)`. BiMap: two maps (`forward`, `backward`) kept in sync, rejecting duplicate values. Guava's `Multimap`, `Multiset` and `BiMap` package these.

Design choice for a multimap: `List` **values** keep duplicates and insertion order, `Set` **values** de-duplicate. Removing the last value of a key should also remove the key (`if (list.isEmpty()) map.remove(key)`), or `containsKey` lies about emptiness. For concurrent use, `ConcurrentHashMap` with `computeIfAbsent(k, x -> new CopyOnWriteArrayList<>())` is the usual recipe.

```java
Map<String, List<String>> multimap = new TreeMap<>();
for (String[] kv : new String[][]{{"fruit", "apple"}, {"veg", "kale"}, {"fruit", "fig"}})
    multimap.computeIfAbsent(kv[0], k -> new ArrayList<>()).add(kv[1]);
System.out.println(multimap);
Map<String, Set<String>> setMultimap = new TreeMap<>();
setMultimap.computeIfAbsent("a", k -> new TreeSet<>()).add("x");
setMultimap.computeIfAbsent("a", k -> new TreeSet<>()).add("x");                    // a Set
per key ignores duplicates
System.out.println(setMultimap);

class Bag<T> {                                                                      // a
multiset: element -> count
```


© TechTribers. For personal use only. Do not redistribute.

```
    private final Map<T, Integer> counts = new TreeMap<>();
    void add(T t) { counts.merge(t, 1, Integer::sum); }
    int count(T t) { return counts.getOrDefault(t, 0); }
    @Override public String toString() { return counts.toString(); }
}
Bag<String> bag = new Bag<>();
for (String s : "a b a c a b".split(" ")) bag.add(s);
System.out.println(bag + " " + bag.count("a") + " " + bag.count("z"));

class BiMap<K, V> {                                                    // both
directions, unique values
    private final Map<K, V> forward = new HashMap<>();
    private final Map<V, K> backward = new HashMap<>();
    void put(K k, V v) {
        if (backward.containsKey(v) && !backward.get(v).equals(k)) throw new
IllegalArgumentException("value already mapped: " + v);
        V old = forward.put(k, v);
        if (old != null) backward.remove(old);
        backward.put(v, k);
    }
    V get(K k) { return forward.get(k); }
    K inverse(V v) { return backward.get(v); }
}
BiMap<String, Integer> codes = new BiMap<>();
codes.put("IN", 91); codes.put("US", 1);
System.out.println(codes.get("IN") + " " + codes.inverse(1));
try { codes.put("UK", 91); } catch (IllegalArgumentException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
{fruit=[apple, fig], veg=[kale]}
{a=[x]}
{a=3, b=2, c=1} 3 0
91 US
value already mapped: 91
```

⚠️ **Interview trap**

`computeIfAbsent(k, x -> new ArrayList<>()).add(v)` is atomic for the *creation* of the list only; adding to a plain `ArrayList` from several threads is still unsafe.

⏳ **Q573** CL-094 • Advanced

Design an in-memory cache with a time-to-live (TTL).

⚡ **Quick answer**

Store each value with its **expiry time** in a `ConcurrentHashMap<K, Entry>`. On `get`, compare the expiry with the clock: if expired, **lazily remove** it (using `remove(key, entry)` so a newer value isn't deleted) and return a miss. Add a **purge** (`values().removeIf(expired)`) on a schedule to reclaim memory for keys that are never read again. Inject the clock so tests can control time.

© TechTribers. For personal use only. Do not redistribute.

Extensions to discuss: a **maximum size** with LRU/LFU eviction (use Caffeine instead of hand-rolling), **cache stampede** protection (`computeIfAbsent` or a per-key future so one loader runs while others wait), background refresh before expiry, **negative caching** (cache "not found" briefly), per-entry TTL, metrics (hit/miss/eviction counts), and a scheduled `purge` via `ScheduledExecutorService`.

`getOrLoad` above can run the loader twice under contention; for strict single-load semantics use `map.compute` (accepting that the bucket stays locked during loading) or a `ConcurrentHashMap<K, CompletableFuture<V>>`.

Java

```
class TtlCache<K, V> {
    private record Entry<V>(V value, long expiresAt) { }
    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
    private final long ttlMillis;
    private final LongSupplier clock;
    TtlCache(long ttlMillis, LongSupplier clock) { this.ttlMillis = ttlMillis; this.clock = clock; }

    void put(K key, V value) { map.put(key, new Entry<>(value, clock.getAsLong() + ttlMillis)); }
    V get(K key) {
        Entry<V> e = map.get(key);
        if (e == null) return null;
        if (e.expiresAt() <= clock.getAsLong()) { map.remove(key, e); return null; }          //
lazy expiry; remove(key, e) never deletes a NEWER value
        return e.value();
    }
    V getOrLoad(K key, Function<K, V> loader) { V v = get(key); if (v != null) return v; v =
loader.apply(key); put(key, v); return v; }
    int purge() { long now = clock.getAsLong(); int before = map.size(); map.values().removeIf(e ->
e.expiresAt() <= now); return before - map.size(); }
    int size() { return map.size(); }
}
AtomicLong now = new AtomicLong(0);                                          // a fake clock makes the
test deterministic
TtlCache<String, String> cache = new TtlCache<>(100, now::get);
cache.put("a", "1"); now.set(60); cache.put("b", "2");
System.out.println(cache.get("a") + " " + cache.get("b"));
now.set(110);                                                                // "a" (stored at t=0) has
expired; "b" (stored at t=60) has not
System.out.println(cache.get("a") + " " + cache.get("b") + " " + cache.size());
now.set(200);
System.out.println(cache.purge() + " " + cache.size());
AtomicInteger loads = new AtomicInteger();
System.out.println(cache.getOrLoad("c", k -> "v" + loads.incrementAndGet()) + " " +
cache.getOrLoad("c", k -> "v" + loads.incrementAndGet()) + " loads=" + loads.get());
```

▶ Output

```
1 2
null 2 1
1 0
v1 v1 loads=1
```


© TechTribers. For personal use only. Do not redistribute.



c is loaded at t=200 by getOrLoad and lives until t=300. remove(key, entry) only deletes the entry you saw, so a newer value survives.

💡 **Pro tip**

Injecting a `LongSupplier` clock is the detail that signals production thinking: it makes expiry testable without `Thread.sleep`.

🚦 **Q574** `CL-095` • Advanced

Implement a sliding-window rate limiter.

⚡ **Quick answer**

Keep a **queue of request timestamps per user**. For each request at time `now`: drop timestamps `<= now - window` from the head, **deny** if `limit` timestamps remain, otherwise append `now` and allow. It's exact (no 2× edge bursts like a fixed window) and uses O(limit) memory per user and amortised O(1) time per request.

Algorithm	Memory/user	Accuracy	Notes
Fixed window counter	O(1)	Allows up to 2× the limit around a window boundary	Simplest
Sliding window log (above)	O(limit)	Exact	Needs a deque per user
Sliding window counter	O(1)	Approximate (weights the previous window)	Good compromise
Token bucket	O(1)	Allows controlled bursts	Most common in production
Leaky bucket	O(queue)	Smooths output rate	Traffic shaping

For multi-threaded use, make `allow` `synchronized` per user (or use a `ConcurrentHashMap` with `compute`), and in a cluster keep counters in Redis (`INCR` with expiry, or a Lua script for atomic sliding windows). Evict idle users, or the map grows without bound.

© TechTribers. For personal use only. Do not redistribute.

Java

```
class SlidingWindowLimiter {
    private final int limit;
    private final long windowMs;
    private final Map<String, Deque<Long>> hits = new HashMap<>();
    SlidingWindowLimiter(int limit, long windowMs) { this.limit = limit; this.windowMs = windowMs; }
    boolean allow(String user, long nowMs) {
        Deque<Long> q = hits.computeIfAbsent(user, u -> new ArrayDeque<>());
        while (!q.isEmpty() && q.peekFirst() <= nowMs - windowMs) q.pollFirst();       // drop
timestamps outside the window
        if (q.size() >= limit) return false;
        q.addLast(nowMs);
        return true;
    }
}
SlidingWindowLimiter rl = new SlidingWindowLimiter(3, 1000);
long[] times = {0, 100, 200, 300, 900, 1000, 1100, 1250};
StringBuilder sb = new StringBuilder();
for (long t : times) sb.append(t).append(rl.allow("asha", t) ? ":ok " : ":DENY ");
System.out.println(sb);
System.out.println(rl.allow("ravi", 300));                                          // limits
are tracked per user
```

▶ Output

```
0:ok 100:ok 200:ok 300:DENY 900:DENY 1000:ok 1100:ok 1250:ok
true
```

Sliding-window log: limit 3 requests per 1000 ms. Timestamps older than the window are dropped



t=300 denied (0, 100, 200 inside the window) · t=900 denied · t=1000 allowed (0 dropped) · t=1100 allowed (100 dropped)

Memory is O(limit) timestamps per user. A fixed-window counter is cheaper but allows 2× bursts at window edges.

⚠ **Interview trap**

Use a monotonic time source and inject it for tests, as with the TTL cache. `System.currentTimeMillis()` can jump backwards when the clock is adjusted.

© TechTribers. For personal use only. Do not redistribute.

Q575 CL-096

• Basic

How do you merge two maps, find their common keys and compute the differences?

⚡ **Quick answer**

Merge with `merge(key, value, combiner)` per entry (adding, taking the max, concatenating) or `putAll` (the second map wins). Common keys: `retainAll` on a copy of the key set; keys only in A: `removeAll`; value conflicts: iterate and compare with `containsKey` and `equals`. For an added/removed/changed diff of two snapshots, compare the key sets and then the values.

`Map.equals` ignores order and implementation, so it's the right comparison for "same content". For nested structures (`Map<String, List<...>>`) it compares deeply because `List.equals` does. For large maps, build the key sets once; `containsKey` is O(1) on a `HashMap` but O(log n) on a `TreeMap`.

Java

```
static void show(Object o) { System.out.println(o); }                          // avoids a println
overload-inference quirk with generic collect() results
Map<String, Integer> a = Map.of("x", 1, "y", 2, "z", 3), b = Map.of("y", 10, "z", 20, "w", 30);
Map<String, Integer> sum = new TreeMap<>(a);
b.forEach((k, v) -> sum.merge(k, v, Integer::sum));                            // merge: add when both
have the key
System.out.println(sum);
Set<String> common = new TreeSet<>(a.keySet()); common.retainAll(b.keySet());
Set<String> onlyA = new TreeSet<>(a.keySet()); onlyA.removeAll(b.keySet());
System.out.println(common + " " + onlyA);
Map<String, Integer> prefer = new TreeMap<>(a); prefer.putAll(b);              // putAll: b wins on
conflicts
System.out.println(prefer);
Map<String, Integer> missing = new TreeMap<>();
a.forEach((k, v) -> { if (!b.containsKey(k)) missing.put(k, v); });
System.out.println(missing);
show(Stream.concat(a.entrySet().stream(), b.entrySet().stream())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, Math::max,
TreeMap::new)));        // keep the larger value per key
Map<String, Integer> before = Map.of("a", 1, "b", 2), after = Map.of("b", 3, "c", 4);
// diff two snapshots
Set<String> added = new TreeSet<>(after.keySet()); added.removeAll(before.keySet());
Set<String> removed = new TreeSet<>(before.keySet()); removed.removeAll(after.keySet());
Set<String> changed = new TreeSet<>();
before.forEach((k, v) -> { if (after.containsKey(k) && !after.get(k).equals(v)) changed.add(k); });
System.out.println("added " + added + " removed " + removed + " changed " + changed);
```

▶ **Output**

```
{w=30, x=1, y=12, z=23}
[y, z] [x]
{w=30, x=1, y=10, z=20}
{x=1}
{w=30, x=1, y=10, z=20}
added [c] removed [a] changed [b]
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `merge` with a function that returns `null` removes the key, a neat way to implement "subtract counts and drop zeros".

⏱️ **Q576** CL-097 • Advanced

Design a time-based key-value store (get the value as of a timestamp).

> ⚡ **Quick answer**
> Use `Map<String, TreeMap<Integer, String>>`: for each key, a `TreeMap` of timestamp → value. `set` is an O(log n) insert, and `get(key, t)` returns * *floorEntry(t)* *, the latest version at or before `t`, or an empty result if none. (If timestamps only ever increase, an `ArrayList` with binary search is lighter.)

This is the LeetCode "Time Based Key-Value Store" and the core of **temporal tables**, audit history and MVCC snapshots. Follow-ups: deleting versions older than a retention period (`headMap(cutoff).clear()`), concurrent access (`ConcurrentSkipListMap` per key), and persistence (an append-only log keyed by `(key, timestamp)`).

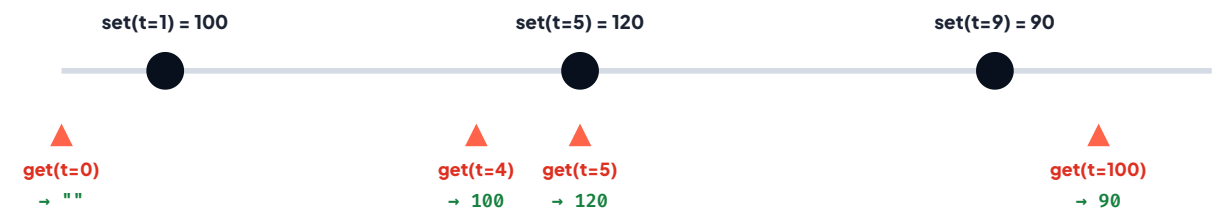
Java

```
class TimeMap {
    private final Map<String, TreeMap<Integer, String>> store = new HashMap<>();
    void set(String key, String value, int timestamp) { store.computeIfAbsent(key, k -> new
TreeMap<>()).put(timestamp, value); }
    String get(String key, int timestamp) {
        TreeMap<Integer, String> versions = store.get(key);
        if (versions == null) return "";
        Map.Entry<Integer, String> e = versions.floorEntry(timestamp);              // latest
version at or before the timestamp: O(log n)
        return e == null ? "" : e.getValue();
    }
}
TimeMap tm = new TimeMap();
tm.set("price", "100", 1); tm.set("price", "120", 5); tm.set("price", "90", 9);
System.out.println(tm.get("price", 0) + "|" + tm.get("price", 1) + "|" + tm.get("price", 4) + "|" +
tm.get("price", 5) + "|" + tm.get("price", 100) + "|" + tm.get("other", 5) + "|");
```

▶ **Output**

```
|100|100|120|90||
```


© TechTribers. For personal use only. Do not redistribute.



floorEntry(t) is the latest version at or before t, in O(log n). Writes in increasing time order could use an array + binary search.

💡 **Pro tip**

The signal word is "as of" or "latest at or before": that's `floorEntry` / `floorKey`. "Earliest at or after" is `ceilingEntry`.

Section 07

Performance, Mistakes & Rapid-Fire

What the complexity numbers mean in practice, the mistakes code reviews catch, and a one-page cheat sheet of the classic differences.

Questions 577–579 · 3 questions

🔎 **Q577** CL-098 • Intermediate

What are the most common collection mistakes found in code reviews?

⚡ **Quick answer**

Using `List.contains` in a loop (O(n·m)), returning `null` instead of an empty collection, iterating `keySet()` and calling `get`, exposing an internal mutable collection, mutating a collection while iterating it, using mutable objects as hash keys, comparing boxed values with `==`, and picking `LinkedList` or `Vector` / `Stack` by habit.

Mistake	Why it bites	Fix
`list.contains` in a loop	Quadratic time	`HashSet` for lookups
Returning `null` collections	NPE for every caller	`List.of()` / `Collections.emptyList()`

© TechTribers. For personal use only. Do not redistribute.

`keySet()` + `get()`	Extra lookups; slower on `TreeMap`	`entrySet()`
Leaking internal collections	Callers mutate your state	`List.copyOf` / unmodifiable view
Modifying during iteration	`ConcurrentModificationException`	`removeIf` , `Iterator.remove`
Mutable hash keys	Entries become unreachable	Immutable keys / records
`==` **on** `Integer` / `Long`	Fails outside -128…127	`equals`
`equals` **without** `hashCode`	Hash lookups miss	Override both, or use a record
Unbounded caches / queues	`OutOfMemoryError`	Size limits, TTL, back-pressure
`LinkedList` , `Vector` , `Stack` **by habit**	Slow, legacy	`ArrayList` , `ArrayDeque`
Sharing `HashMap` / `ArrayList` **between threads**	Lost updates, corruption	`ConcurrentHashMap` , immutable snapshots

Most of these have a dedicated question earlier in Parts 12 and 13; this is the review checklist version.

Java

```java
// 1. contains() inside a loop over a List: O(n * m). Turn the lookup side into a Set
List<Integer> ids = IntStream.range(0, 1_000).boxed().toList(), wanted = List.of(5, 500, 999,
2_000);
Set<Integer> idSet = new HashSet<>(ids);
System.out.println(wanted.stream().filter(idSet::contains).toList());
// 2. returning null instead of an empty collection
static List<String> findNames(boolean any) { return any ? List.of("a") : List.of(); }
System.out.println(findNames(false).size());
// 3. keySet() + get(): an extra lookup per key. Use entrySet()
Map<String, Integer> m = new TreeMap<>(Map.of("a", 1, "b", 2));
int total = 0; for (Map.Entry<String, Integer> e : m.entrySet()) total += e.getValue();
System.out.println(total);
// 4. exposing an internal collection: return a copy or an unmodifiable view
class Team {
    private final List<String> members = new ArrayList<>(List.of("asha"));
    List<String> members() { return List.copyOf(members); }
    void add(String s) { members.add(s); }
}
Team team = new Team();
try { team.members().add("hacker"); } catch (UnsupportedOperationException e) {
System.out.println("members() is read-only"); }
// 5. removing while iterating: use removeIf
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
nums.removeIf(n -> n % 2 == 0);
System.out.println(nums);
```

▶ Output

```
[5, 500, 999]
0
3
members() is read-only
[1, 3, 5]
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> A strong way to answer "what do you look for in a code review?" is to pick three from this table and describe a bug each one caused or could cause.

📊 **Q578** CL-099 • Intermediate

How much faster is HashSet.contains than ArrayList.contains, and when does it matter?

> ⚡ **Quick answer**
> `ArrayList.contains` is O(n): it scans the elements and calls `equals`. `HashSet.contains` is O(1) on average. For 2,000 lookups in a 100,000-element collection, measured on JDK 25: **~430–500 ms** with `ArrayList` vs **~0.2–4 ms** with `HashSet`, a factor of more than 100. For a handful of elements (under ~10–20) a list scan is as fast or faster because of cache locality and no hashing.

Measured here (JDK 25, three rounds; the HashSet numbers vary because they are tiny and include JIT warm-up):

Round	`ArrayList.contains` × 2,000	`HashSet.contains` × 2,000
1	498 ms	1.5 ms
2	434 ms	0.16 ms
3	482 ms	4.2 ms

When it matters: lookups in loops over thousands of items, joins of two collections, de-duplication, and membership filters. When it doesn't: tiny collections, one-off lookups, or when you also need index access and order (keep the list and add a parallel `HashSet` index only if profiling shows the scan is a hotspot).

Cost of the set: about 57 bytes per element versus 20 (see the memory question in Part 12), plus building it (O(n)). Build it once and reuse it.

Java

```java
int n = 100_000;
List<Integer> list = IntStream.range(0, n).boxed().toList();
Set<Integer> set = new HashSet<>(list);
int hitsList = 0, hitsSet = 0;
for (int i = 0; i < 2_000; i++) {
    if (list.contains(n - 1 - i)) hitsList++;                    // scans up to 100,000 elements
each time
    if (set.contains(n - 1 - i)) hitsSet++;                      // one hash computation and a
bucket check
}
System.out.println(hitsList + " " + hitsSet);                    // same answer: only the cost
differs
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
2000 2000
```

⚠️ **Interview trap**

Micro-benchmarks written as plain loops mislead (JIT warm-up, dead-code elimination). Use JMH for decisions that matter; these numbers show the order of magnitude, not a benchmark result.

📋 **Q579** `CL-100` • Basic

Rapid-fire: the classic collection differences in one table.

⚡ **Quick answer**

The distinctions interviewers ask most often, with the one-line answer for each. Details and code for every row are in Parts 3, 12 and 13.

Question	One-line answer
`ArrayList` vs `LinkedList`	Array with O(1) index vs doubly-linked nodes with O(1) ends; `ArrayList` wins almost always
`ArrayList` vs `Vector`	`Vector` is synchronised and legacy; use `ArrayList` (+ `CopyOnWriteArrayList` if shared)
`HashMap` vs `Hashtable`	`Hashtable` : synchronised, no nulls, legacy; `HashMap` : unsynchronised, one null key
`HashMap` vs `ConcurrentHashMap`	Bucket-level locking and atomic compound operations, no nulls, weakly consistent iterators
`HashMap` vs `LinkedHashMap` vs `TreeMap`	Unordered O(1) vs insertion/access order O(1) vs sorted O(log n)
`HashSet` vs `TreeSet`	Hash table (backed by `HashMap`) vs red-black tree (backed by `TreeMap`); sorted needs `Comparable`
`Comparable` vs `Comparator`	Natural order inside the class vs external, multiple orders
`Iterator` vs `ListIterator` vs `Enumeration`	Forward + remove; bidirectional + add/set (lists); legacy read-only
Fail-fast vs fail-safe	CME (best-effort) vs snapshot or weakly consistent iterators
`Collection` vs `Collections`	Root interface vs static utility class
`List.of` vs `Arrays.asList` vs `unmodifiableList`	Immutable, no nulls vs fixed-size write-through view vs read-only view of a mutable list
`poll` vs `remove` **(Queue)**	Returns `null` on empty vs throws
`PriorityQueue` **order**	Heap order: only `poll()` is sorted, not iteration
`ArrayDeque` vs `Stack`	Fast, unsynchronised, no nulls vs legacy `Vector` subclass

© TechTribers. For personal use only. Do not redistribute.

`equals` / `hashCode` **contract**	Equal objects have equal hashes; override both; use immutable keys
`HashMap` **resize**	At size > 0.75 × capacity; table doubles; lazily allocated at 16
Tree bin threshold	8 nodes in a bucket and table ≥ 64; back to a list at 6
`WeakHashMap` **vs** `IdentityHashMap`	Weak keys cleared by GC vs keys compared with `==`
`CopyOnWriteArrayList` **use case**	Read-mostly small lists; iterator is a snapshot; writes copy the array
`ArrayBlockingQueue` **vs** `LinkedBlockingQueue`	Fixed capacity, one lock vs optional capacity, two locks
`Collectors.toMap` **vs** `groupingBy`	One value per key (duplicates throw) vs lists/aggregates per key
`groupingBy` **vs** `partitioningBy`	Any number of groups, absent if empty vs exactly two, always present

💡 **Pro tip**

Don't memorise the table; memorise the *reason* behind each row (the data structure underneath). Every answer follows from "what is it made of, and what does that cost?".

© TechTribers. For personal use only. Do not redistribute.

Series progress

579 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 14 starts at Q580.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.